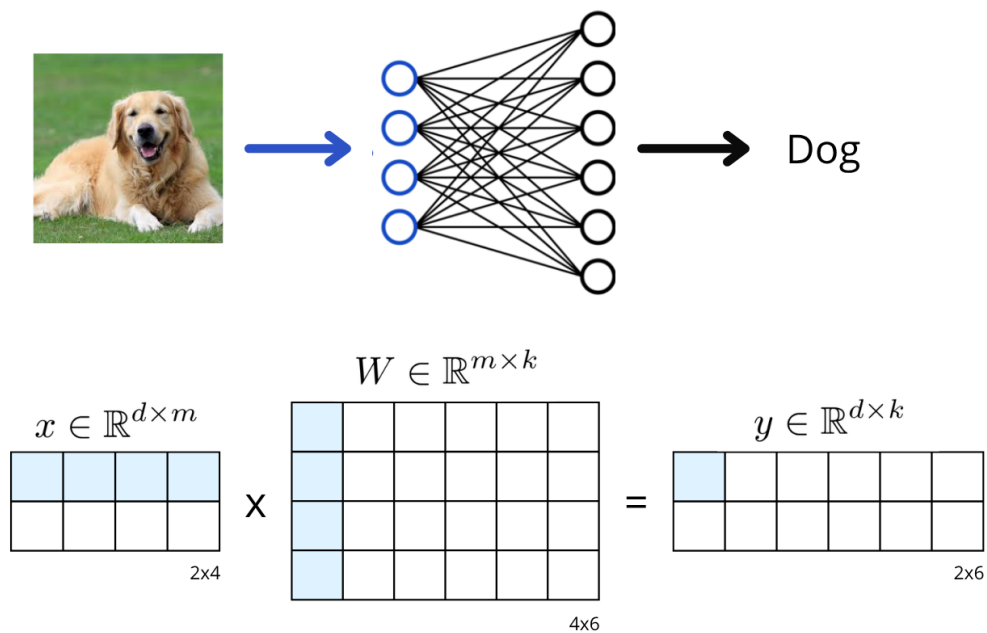


---

# Matrix Multiplication Free Model for Image Classification

---



**AIS4900 Master Thesis in Mechatronics and Automation**

Jon Urcelay San Roman, 20th May 2026

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>3</b>  |
| 1.1      | Background and Related Work . . . . .                            | 3         |
| 1.2      | Project Objectives . . . . .                                     | 5         |
| 1.3      | Report Structure . . . . .                                       | 5         |
| <b>2</b> | <b>Theoretical background</b>                                    | <b>6</b>  |
| 2.1      | Convolutional Neural Network (CNN) . . . . .                     | 6         |
| 2.1.1    | Convolutional layer . . . . .                                    | 6         |
| 2.1.2    | Pooling layer . . . . .                                          | 7         |
| 2.1.3    | Fully-connected layer . . . . .                                  | 8         |
| 2.2      | YOLOv1 . . . . .                                                 | 8         |
| 2.2.1    | Detection Explained . . . . .                                    | 8         |
| 2.2.2    | Network Design . . . . .                                         | 10        |
| 2.2.3    | Training . . . . .                                               | 10        |
| 2.2.4    | Inference . . . . .                                              | 11        |
| 2.3      | Matrix Multiplication Free (MMF) Models . . . . .                | 12        |
| <b>3</b> | <b>Methods</b>                                                   | <b>14</b> |
| 3.1      | Dataset . . . . .                                                | 14        |
| 3.1.1    | Data Augmentation . . . . .                                      | 15        |
| 3.2      | Model Architectures and Implementation Details . . . . .         | 16        |
| 3.2.1    | Baseline Model Architecture . . . . .                            | 17        |
| 3.2.2    | Baseline Model Implementation Details . . . . .                  | 17        |
| 3.2.3    | MMF Model Architecture . . . . .                                 | 17        |
| 3.2.4    | MMF Model Implementation Details on Dedicated Hardware . . . . . | 19        |
| 3.2.5    | MMF Model Implementation Details on GPU . . . . .                | 19        |
| 3.3      | Training Procedure . . . . .                                     | 21        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 3.4      | Evaluation Metrics . . . . .                    | 22        |
| 3.5      | Hardware and Software Environment . . . . .     | 23        |
| <b>4</b> | <b>Results and Discussion</b>                   | <b>24</b> |
| 4.1      | Experiment Plan . . . . .                       | 24        |
| 4.2      | Baseline Model . . . . .                        | 24        |
| 4.2.1    | Hyperparameter Tuning . . . . .                 | 25        |
| 4.2.2    | Baseline Model with Ternary Inference . . . . . | 27        |
| 4.3      | MMF Model . . . . .                             | 28        |
| 4.3.1    | Hyperparameter Tuning . . . . .                 | 28        |
| 4.3.2    | Modifying Number of Channels . . . . .          | 30        |
| 4.3.3    | Modifying Weight Initialization Scale . . . . . | 31        |
| 4.3.4    | Ternary, Quinary, Septenary, Nonary . . . . .   | 32        |
| 4.4      | Best Model Comparison . . . . .                 | 33        |
| 4.4.1    | Configuration Summary . . . . .                 | 33        |
| 4.4.2    | Training Comparison . . . . .                   | 33        |
| 4.4.3    | Overall Performance Comparison . . . . .        | 35        |
| 4.4.4    | Per Class Performance Comparison . . . . .      | 36        |
| 4.4.5    | Weight Distribution Comparison . . . . .        | 39        |
| <b>5</b> | <b>Conclusion</b>                               | <b>40</b> |
| <b>6</b> | <b>Future Work</b>                              | <b>42</b> |
|          | <b>References</b>                               | <b>44</b> |
| <b>A</b> | <b>Code Repository Structure</b>                | <b>46</b> |
| <b>B</b> | <b>Complete Training Results</b>                | <b>47</b> |
| B.1      | Baseline Model: Hyperparameter Tuning . . . . . | 47        |

|     |                                                            |    |
|-----|------------------------------------------------------------|----|
| B.2 | MMF Model: Hyperparameter Tuning . . . . .                 | 49 |
| B.3 | MMF Model: Modifying Number of Channels . . . . .          | 51 |
| B.4 | MMF Model: Modifying Weight Initialization Scale . . . . . | 53 |
| B.5 | MMF Model: Ternary, Quinary, Septenary, Nonary . . . . .   | 55 |



## List of Figures

|    |                                                                                                                                                                                                |    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Overview of the layers of a CNN. . . . .                                                                                                                                                       | 6  |
| 2  | Visual representation of a kernel (left) and its stride (right) in a CNN. . . . .                                                                                                              | 7  |
| 3  | Visual representation of a pooling types in a CNN. . . . .                                                                                                                                     | 8  |
| 4  | YOLOv1 detection process. The image is divided into an $S \times S$ grid. Each cell predicts $B$ bounding boxes and confidence scores, as well as $C$ class probabilities (from [11]). . . . . | 9  |
| 5  | YOLOv1 network architecture (from [11]). . . . .                                                                                                                                               | 10 |
| 6  | Sample images from CIFAR-10 dataset, showing one row per class. . . . .                                                                                                                        | 14 |
| 7  | Validation Accuracy while Training for selected runs from Baseline Model hyperparameter tuning . . . . .                                                                                       | 25 |
| 8  | Validation Accuracy while Training for chosen Hyperparameter Tuning Experiments with MMF Model . . . . .                                                                                       | 29 |
| 9  | Learning Rate while Training for Baseline vs Best MMF Model . . . . .                                                                                                                          | 34 |
| 10 | Training Accuracy while Training for Baseline vs Best MMF Model . . . . .                                                                                                                      | 34 |
| 11 | Validation Accuracy while Training for Baseline vs Best MMF Model . . . . .                                                                                                                    | 35 |
| 12 | Average Confidence Score for Correct and Incorrect Predictions on Validation Data for Baseline vs Best MMF Model . . . . .                                                                     | 36 |
| 13 | Confusion Matrices for Baseline Model . . . . .                                                                                                                                                | 37 |
| 14 | Confusion Matrices for Best MMF Model . . . . .                                                                                                                                                | 37 |
| 15 | Baseline vs Best MMF Model Per Class Top-1 Accuracy . . . . .                                                                                                                                  | 38 |
| 16 | Baseline vs Best MMF Model Per Class Top-3 Accuracy . . . . .                                                                                                                                  | 38 |
| 17 | Baseline vs Best MMF Model Per Class Top-5 Accuracy . . . . .                                                                                                                                  | 39 |
| 18 | Weight Distribution for Baseline vs Best MMF Model . . . . .                                                                                                                                   | 39 |
| 19 | Validation Accuracy while Training for all Hyperparameter Tuning Experiments with Baseline Model . . . . .                                                                                     | 47 |
| 20 | Validation Accuracy while Training for all Hyperparameter Tuning Experiments with MMF Model . . . . .                                                                                          | 49 |
| 21 | Validation Accuracy while Training for all Number of Channel Experiments with MMF Model . . . . .                                                                                              | 51 |

|    |                                                                                                                     |    |
|----|---------------------------------------------------------------------------------------------------------------------|----|
| 22 | Validation Accuracy while Training for all Experiments Modifying Weight Initialization with MMF Model . . . . .     | 53 |
| 23 | Validation Accuracy while Training for all Experiments for Ternary, Quinary, Septenary, Nonary MMF Models . . . . . | 55 |

## List of Tables

|    |                                                                                                                                                                                                                                                                                         |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Augmentation pipelines used across experiments. A dash indicates the transformation is not applied. . . . .                                                                                                                                                                             | 16 |
| 2  | Detailed layer configuration of the model architecture (shared by both baseline and MMF models). . . . .                                                                                                                                                                                | 18 |
| 3  | Summary of all experiment groups conducted in this project. . . . .                                                                                                                                                                                                                     | 24 |
| 4  | Selected runs from Baseline Model hyperparameter tuning. BS: batch size, $\eta$ : learning rate, $\lambda$ : weight decay, Aug: augmentation level (W: weak, S: strong, S+N: strong with normalization). . . . .                                                                        | 25 |
| 5  | Best Baseline Model configuration (Run 42). . . . .                                                                                                                                                                                                                                     | 27 |
| 6  | Comparison of the Baseline Model under standard and MMF-style inference. . .                                                                                                                                                                                                            | 27 |
| 7  | Selected runs from MMF Model hyperparameter tuning. BS: batch size, $\eta$ : learning rate, $\lambda$ : weight decay. . . . .                                                                                                                                                           | 28 |
| 8  | Results for channel scaling experiments. . . . .                                                                                                                                                                                                                                        | 30 |
| 9  | Results for weight initialization scale experiments. $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization. . . . .                                                                                                                                     | 31 |
| 10 | Results for quantization level experiments. W Init: weight initialization scale. $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization. . . . .                                                                                                         | 32 |
| 11 | Best training configurations for Baseline and MMF models. . . . .                                                                                                                                                                                                                       | 33 |
| 12 | Performance comparison between best Baseline and MMF models. . . . .                                                                                                                                                                                                                    | 35 |
| 13 | Top-K Validation Accuracy Comparison on CIFAR-10 Validation Set. . . . .                                                                                                                                                                                                                | 35 |
| 14 | Complete results from Baseline Model hyperparameter tuning. BS: batch size, LR: learning rate, $\lambda$ : weight decay, Aug: augmentation level (W: weak, S: strong, S+N: strong with normalization), Sch: scheduler (ROP: ReduceLROnPlateau, OCL: OneCycleLR), Pat: patience. . . . . | 48 |

|    |                                                                                                                                                                                                                                                        |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 15 | Complete results from MMF Model hyperparameter tuning (Runs 0–34). All runs use batch size 1024. LR: learning rate, $\lambda$ : weight decay, Sch: scheduler (RLROP: ReduceLROnPlateau, Cos: CosineAnnealingLR). . . . .                               | 50 |
| 16 | Complete results for channel scaling experiments. The reference MMF Model ( $\alpha = 1.0$ , Run 20) is included for comparison. . . . .                                                                                                               | 52 |
| 17 | Complete results for weight initialization scale experiments. The reference MMF Model (scale = 1.0, Run 20) is included for comparison. $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization. . . . .                 | 54 |
| 18 | Complete results for quantization level experiments. The ternary reference ( $k = 1$ ) is included for comparison. W Init: weight initialization scale. $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization. . . . . | 56 |

## Acronyms

**PTQ** Post-Training Quantization.

**QAT** Quantization-Aware Training.

**INT** Integer.

**FP** Floating Point.

**STE** Straight-Through Estimator.

**TWN** Ternary Weight Network.

**TTQ** Trained Ternary Quantization.

**LLM** Large Language Model.

**DNN** Deep Neural Network.

**NN** Neural Network.

**CNN** Convolutional Neural Network.

**RGB** Red Green Blue.

**ReLU** Rectified Linear Unit.

**FC** Fully Connected.

**YOLO** You Only Look Once.

**IoU** Intersection Over Union.

**HSV** Hue Saturation Value.

**NMS** Non-Maximum Suppression.

**MMF** Matrix Multiplication Free.

**CPU** Central Processing Unit.

**GPU** Graphics Processing Unit.

**ASIC** Application-Specific Integrated Circuit.

**FPGA** Field-Programmable Gate Array.

**RMS** Root Mean Square.

**HBM** High Bandwidth Memory.

**RAM** Random Access Memory.

**VRAM** Video Random Access Memory.

**PP** Percentage Points.

**LR** Learning Rate.

**WD** Weight Decay.

## Abstract

Neural networks rely heavily on floating-point multiplications, which are computationally more expensive than addition/subtraction operations. In this way, Matrix Multiplication Free (MMF) networks replace multiplications with simple additions/subtractions, offering the potential for significantly lowering energy consumption, but at the cost of reduced representational capacity.

This project investigates the feasibility of MMF networks applied to image classification. For this, two architecturally equivalent models are designed and trained under identical conditions: a full-precision Matrix Multiplication Model and a MMF Model. Then, with the goal of improving results for the MMF Model, several experimental runs are conducted covering different optimization techniques.

Final results show that MMF convolutional networks are trainable and achieve non-trivial classification accuracy. However, closing the performance gap with full-precision models remains an open challenge that motivates further architectural innovation.

# 1 Introduction

Modern deep neural networks (DNNs) for computer vision have achieved remarkable performance across a wide range of tasks, from image classification to object detection and semantic segmentation. This progress has been driven largely by scaling model size, dataset size, and compute, following empirical scaling laws that show consistent improvements as these quantities grow [1].

However, this performance comes at a computational cost: the dominant operation in every modern DNN is floating-point matrix multiplication (MatMul), which accounts for the vast majority of arithmetic operations during both training and inference. As models continue to grow in size and are increasingly deployed on resource-constrained hardware, like edge devices, embedded systems, and mobile platforms, the energy consumption, memory bandwidth, and silicon area requirements of floating-point MatMul become a fundamental bottleneck.

This tension between model performance and computational efficiency has motivated a substantial body of research into network quantization: the replacement of high-precision floating-point weights with low-bit integer representations, eliminating floating-point multiplications entirely and replacing them with additions and subtractions, which are cheaper on both general-purpose and dedicated hardware.

In this way, this project researches whether this quantization paradigm, formalized as the Matrix Multiplication Free (MatMul-Free or MMF) framework, can be successfully applied to convolutional neural networks (CNNs) for image classification.

## 1.1 Background and Related Work

### Network quantization

Quantization of neural network weights and activations has been studied extensively with the goal of reducing inference costs. There are two main approaches for quantization:

- **Post-Training Quantization (PTC)**. This approach takes an already trained neural network and quantizes it without retraining from scratch.
- **Quantization-Aware Training (QAT)**. This approach includes quantization as an integral part of the training process of the network.

Fixed-point quantization to 8 bits (INT8) is now standard practice in production inference pipelines, supported natively by modern hardware accelerators such as NVIDIA Tensor Cores and Google TPUs, and has been shown to incur negligible accuracy loss on most tasks [2].

However, at lower bit widths, like INT4 or INT2, the quantization error introduced by the coarser representation begins to significantly affect model accuracy, and more careful training procedures are required to recover performance.

## Binary and ternary neural networks

The extreme end of the quantization spectrum restricts weights to 1 bit ( $\{-1, +1\}$ , binary) or 2 bits ( $\{-1, 0, +1\}$ , ternary).

BinaryConnect [3] first demonstrated that binary weights could be used during the forward and backward passes while maintaining full-precision latent weights for the gradient update, using the straight-through estimator (STE) [4] to propagate gradients through the non-differentiable rounding operation.

Ternary Weight Networks (TWN) [5] showed that allowing a third zero value in the weight representation substantially improves accuracy over binary networks, as the zero weights effectively implement a form of structured sparsity.

Despite these advances, binary and ternary convolutional networks consistently lag behind their full-precision counterparts by a significant margin on challenging benchmarks, and their adoption in production systems has been limited.

## BitNet and 1-bit large language models

The landscape changed significantly with the introduction of BitNet [6], which demonstrated that Transformer-based large language models (LLMs) could be trained from scratch with 1-bit weights using a QAT approach, achieving competitive perplexity scores while reducing memory and inference cost substantially. BitNet replaces the standard `nn.Linear` layer with a `BitLinear` layer that quantises weights to  $\{-1, +1\}$  and activations to 8-bit integers before each matrix multiplication.

BitNet b1.58 [7] extended this to ternary weights ( $\{-1, 0, +1\}$ ), observing that the additional zero value significantly improves model quality while retaining the key efficiency properties. Remarkably, BitNet b1.58 models with sufficient parameter counts were shown to match the performance of full-precision LLMs of equivalent size on a range of language understanding benchmarks, while requiring significantly less memory and enabling faster inference through the elimination of floating-point multiplications. These results established ternary weight quantization as a practically viable approach for large-scale language modeling, and motivated the question of whether similar gains could be achieved in other domains.

## The MatMul-Free framework

Building directly on the insights of BitNet b1.58, Zhu et al. [8] proposed the MatMul-Free (MMF) framework, which eliminates matrix multiplications entirely from the forward pass of a neural network. The key innovation is the `MMFLinear` layer, replacing the standard linear transformation with a sequence of additions and negations on integer-valued tensors.

The authors demonstrate that language models built with MMF layers can match the performance of standard Transformer models on language modelling benchmarks, while offering significant theoretical advantages in terms of hardware efficiency.

While the MMF framework has been validated in the language modelling domain, its extension to convolutional neural networks for computer vision presents additional challenges. To the

best of knowledge, no prior work has systematically investigated the application of the MMF framework to convolutional image classification, making this project a novel contribution in this direction.

## 1.2 Project Objectives

This project studies the feasibility of adapting the MMF framework to image classification using convolutional neural networks. Concretely, two architecturally equivalent models will be designed and trained on the CIFAR-10 benchmark dataset [9]: a full-precision Baseline Model and a MMF Model. Both models will be trained under identical conditions and compared on the same metrics, as specified in this project.

The specific research questions this project aims to answer are:

- Can a MMF convolutional network be trained stably on CIFAR-10?
- What hyperparameter considerations are specific to the MMF setting?
- What is the accuracy gap between the baseline model and the MMF Model?
- What are the primary causes for such gap?
- Is there any targeted modification that can reduce this gap?

## 1.3 Report Structure

The remainder of this report is organized as follows. Section 3 describes the dataset, model architectures, implementation details, training procedure, and evaluation metrics. Section 4 presents and analyses all experimental results, organized by experiment group. Section 5 summarizes the main findings and conclusions of the project, and Section 6 outlines directions for future research.

The full source code is publicly available in github <sup>1</sup>, which is described in detail in Appendix A.

---

<sup>1</sup>[https://github.com/jonurce/MatMulFree\\_Image\\_Classification\\_Model](https://github.com/jonurce/MatMulFree_Image_Classification_Model)



## 2 Theoretical background

### 2.1 Convolutional Neural Network (CNN)

CNNs [10] are distinguished from other NN by their superior performance with image, speech or audio signal inputs. They are more often used for classification, object recognition, segmentation or other computer vision tasks.

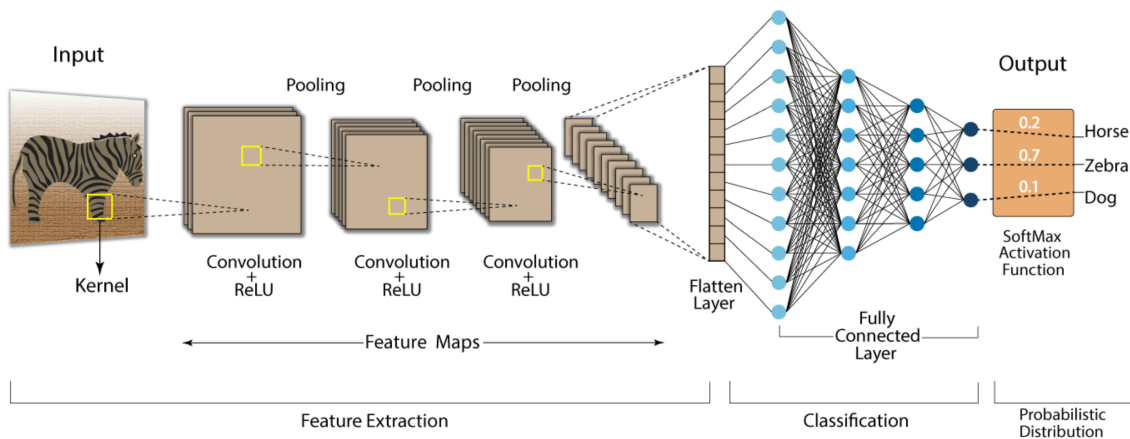


Figure 1: Overview of the layers of a CNN.

They have three main types of layers (Figure 1). The convolutional layer is the first layer, which can be followed by additional convolutional layers or pooling layers, leading to the final fully-connected layer. These are explained below:

#### 2.1.1 Convolutional layer

The convolutional layer is the core building block of a CNN. It requires input data, a filter (also known as feature detector or kernel) and a feature map (also known as activation map or a convolved feature).

- The input data is generally structured in 3-D, like an RGB image, which is a 3-D matrix of pixels (height, width and channel).
- The filter (Figure 2) is a matrix of weights that represents part of the image, typically a 3x3 matrix. It goes moving across the receptive fields of the image, calculating a dot product between the input pixels and the weights at each time, which is fed into an output array. In each step, the filter shifts by a stride, repeating the process until the kernel has swept across the entire image. Each of this steps is known as a convolution.
- The feature map is the final output from the series of dot products, to which a ReLU transformation is applied to introduce nonlinearity to the model.

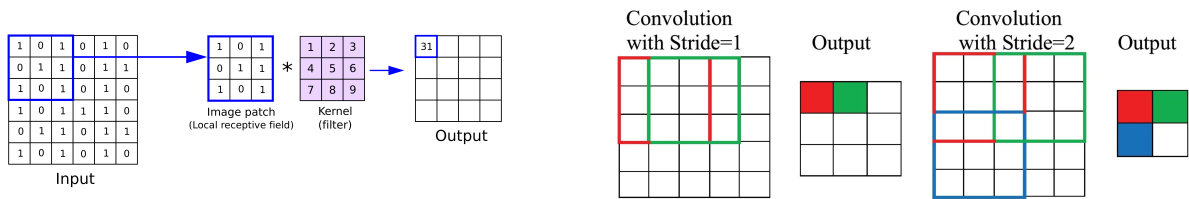


Figure 2: Visual representation of a kernel (left) and its stride (right) in a CNN.

Note that the weights in the feature detector remain fixed as it moves across the image (this is known as parameter sharing), which are adjusted during training (backpropagation and gradient descent). However, there are three hyperparameters that need to be set before training:

1. Number of filters: affects the depth of the output. For example, three distinct filters would yield three different feature maps, creating a depth of three.
2. Stride: number of pixels that the kernel moves at each convolution step (see Figure 2). A larger stride yields a smaller output.
3. Padding: used when the filters do not fit the input image. Three types:
  - Valid/no padding: the last convolution is dropped if dimensions do not align.
  - Same padding: ensures that the output layer has the same size as the input layer.
  - Full padding: increases the size of the output by adding zeros to the border of the input.

After an initial convolution layer, another additional convolution layer can follow. When this happens, the structure of the CNN becomes hierarchical with respect to features, as later layers can see the pixels within the receptive fields of prior layers. Earlier layers focus on simple features, such as colors and edges, while later layers recognize larger elements or shapes of the object, until the final layer identifies the intended object.

### 2.1.2 Pooling layer

The pooling or downsampling layer, conducts dimensionality reduction. In the same way as the convolutional layer, it sweeps a filter across the entire input, but this time the filter does not have any weights. Instead, the kernel applies an aggregation function to its input values. There are two main types of pooling, which define the aggregation function to be used (Figure 3):

- Max pooling: the pixel with the maximum value is sent to the output array.
- Average pooling: the average value of the input pixels is sent to the output array.

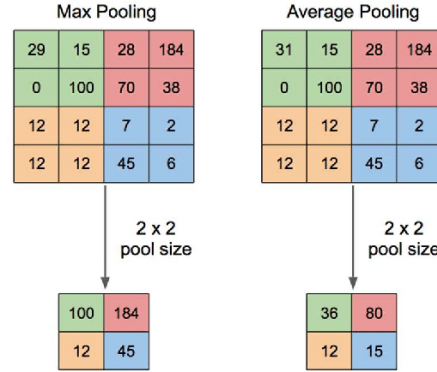


Figure 3: Visual representation of a pooling types in a CNN.

After the aggregation function, a ReLU activation function is often performed. While a lot of information is lost in the pooling layer, it also helps to reduce complexity, improve efficiency, and limit risk of overfitting.

### 2.1.3 Fully-connected layer

The Fully-connected (FC) layer, each node in the output layer connects directly to all nodes in the previous layer. This layer performs the task of classification based on the features extracted through the previous layers and their different filters. FC layers usually leverage a softmax activation function for classification tasks, to produce a probability from 0 to 1.

## 2.2 YOLOv1

You Only Look Once (YOLO) models are a family of neural networks that frame object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities, in a single forward pass. Its first version, YOLOv1 [11], introduced this unified approach, making real-time object detection possible.

### 2.2.1 Detection Explained

The system divides the input image into an  $S \times S$  grid. Each grid cell is responsible for detecting objects whose center falls within that cell. Every grid cell predicts  $B$  bounding boxes along with confidence scores for those boxes. The confidence score reflects both the probability that an object is present in the box and how accurate the box coordinates are. It is formally defined as:

$$\text{Confidence} = \text{Pr}(\text{Object}) \times \text{IOU}_{\text{truth}}^{\text{pred}} \quad (1)$$

If no object exists in a grid cell, the confidence score should be zero. Otherwise, it should equal the Intersection over Union (IoU) between the predicted box and the ground truth box.

Each bounding box consists of five predictions:  $(x, y, w, h, \text{confidence})$ . The coordinates  $(x, y)$  represent the center of the box relative to the grid cell, while the width  $(w)$  and height  $(h)$  are predicted relative to the entire image.

In addition to bounding boxes, each grid cell predicts  $C$  conditional class probabilities,  $\Pr(\text{Class}_i | \text{Object})$ . These probabilities are conditioned on the grid cell containing an object. Only one set of class probabilities is predicted per grid cell, regardless of the number of bounding boxes  $B$ .

At test time, the conditional class probabilities and the box confidence scores are multiplied to produce class-specific confidence scores for each bounding box:

$$\Pr(\text{Class}_i | \text{Object}) \times \Pr(\text{Object}) \times \text{IOU}_{\text{truth}}^{\text{pred}} = \Pr(\text{Class}_i) \times \text{IOU}_{\text{truth}}^{\text{pred}} \quad (2)$$

These scores encode both the probability of a particular class being present in the box and how well the predicted box fits the object.

The final output of the network is a tensor of shape  $S \times S \times (B \times 5 + C)$ . As a reference, for evaluation on the PASCAL VOC dataset [12], the model uses  $S = 7$ ,  $B = 2$ , and  $C = 20$  (corresponding to the 20 object classes in VOC). This results in a final prediction tensor of size  $7 \times 7 \times 30$ .

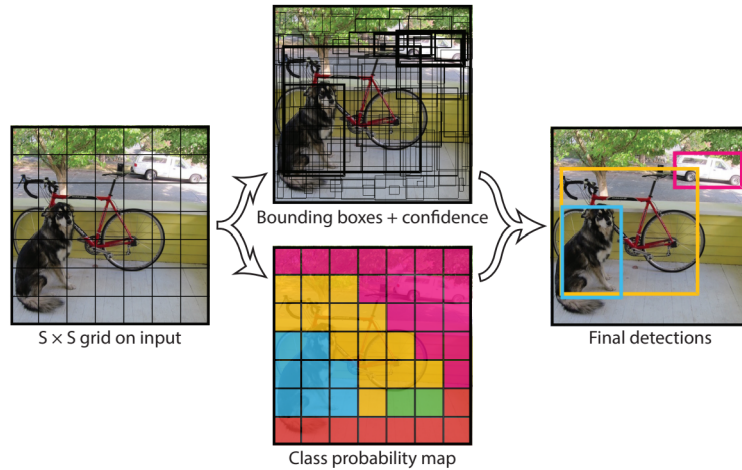


Figure 4: YOLOv1 detection process. The image is divided into an  $S \times S$  grid. Each cell predicts  $B$  bounding boxes and confidence scores, as well as  $C$  class probabilities (from [11]).

## 2.2.2 Network Design

The network architecture contains 24 convolutional layers followed by 2 fully connected layers. The initial convolutional layers of the network extract features from the image while the fully connected layers predict the output probabilities and coordinates. The full network is shown in Figure 5.

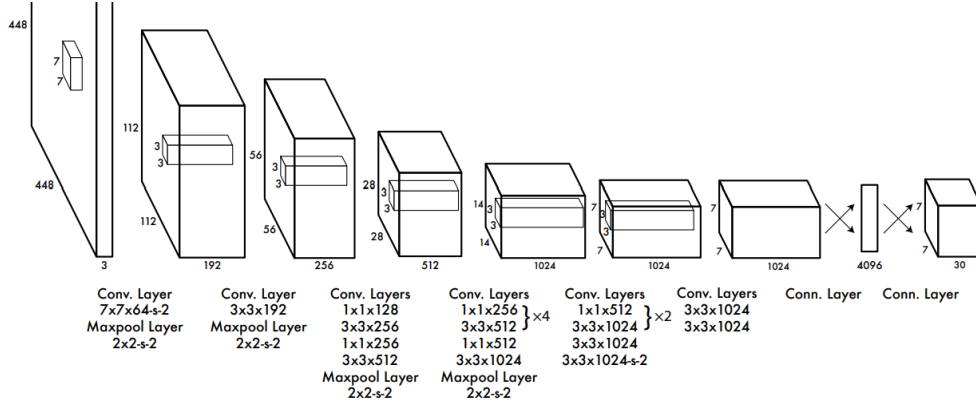


Figure 5: YOLOv1 network architecture (from [11]).

## 2.2.3 Training

YOLOv1 uses a two-stage training procedure.

First, the first 20 convolutional layers from the network are pre-trained on the ImageNet 1000-class classification dataset [13]. These layers are followed by an average-pooling layer and a fully connected layer, so that the output of the network is defined by 1000 values, each of them representing the probability of the image being of that class. After training, this network achieves an accuracy of 88% on the ImageNet 2012 validation set.

Second, the model is converted for the object detection task. This is done by adding, on top of the pre-trained backbone, four convolutional layers and two fully connected layers with randomly initialized weights. In addition, to capture fine-grained visual information required for object detection, the input resolution is increased from  $224 \times 224$  to  $448 \times 448$ . As explained in section 2.2.1, the final layer predicts both class probabilities and bounding box coordinates.

The final layer uses a linear activation function, while all other layers use the leaky rectified linear activation function, defined as:

$$\phi(x) = \begin{cases} x, & \text{if } x > 0 \\ 0.1x, & \text{otherwise} \end{cases} \quad (3)$$

The model is optimized using sum-squared error on the output. Although easy to optimize, sum-squared error does not perfectly align with the goal of maximizing average precision, as it

weights localization and classification errors equally and can cause instability due to many grid cells without objects. To address this, the authors introduce two scaling parameters:  $\lambda_{\text{coord}} = 5$  for bounding box coordinate predictions and  $\lambda_{\text{noobj}} = 0.5$  for confidence predictions of boxes without objects. Additionally, the square root of the width and height is predicted instead of the raw values to reduce the relative impact of errors in large boxes compared to small ones.

During training, only one bounding box predictor per grid cell is assigned responsibility for each object, the one with the highest Intersection over Union (IoU) with the ground truth. This encourages specialization among the predictors.

The complete multi-part loss function optimized by YOLOv1 is:

$$\begin{aligned}
& \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} \left[ \left( \sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left( \sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbf{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned} \tag{4}$$

where  $\mathbf{1}_i^{\text{obj}}$  indicates if an object appears in cell  $i$ , and  $\mathbf{1}_{ij}^{\text{obj}}$  denotes that the  $j$ -th bounding box predictor in cell  $i$  is responsible for the prediction.

The network is trained for 135 epochs on the PASCAL VOC 2007 and 2012 datasets. Training used a batch size of 64, momentum of 0.9, and weight decay of 0.0005. The learning rate schedule starts by gradually increasing from  $10^{-3}$  to  $10^{-2}$ , then remained at  $10^{-2}$  for 75 epochs,  $10^{-3}$  for 30 epochs, and finally  $10^{-4}$  for 30 epochs. To prevent overfitting, dropout with rate 0.5 after the first fully connected layer is used, as well as extensive data augmentation (random scaling and translations up to 20% of the original image size, and randomly adjusted exposure and saturation by up to a factor of 1.5 in the HSV color space).

## 2.2.4 Inference

At inference time, YOLOv1 requires only a single forward pass through the network to predict detections for an entire image.

For the PASCAL VOC dataset, the network predicts a total of 98 bounding boxes per image ( $7 \times 7$  grid  $\times$  2 boxes per cell) along with their corresponding class probabilities. To eliminate redundant detections, the next post-processing steps are applied, resulting in improved final predictions:

1. **Decoding & filtering:** Raw predictions are decoded into bounding boxes, objectness scores, and class probabilities. Low-confidence candidates, defined by  $\Pr(\text{Class}_i | \text{Object}) \times \Pr(\text{Object}) < \text{threshold}$ , are discarded to reduce noise.
2. **Non-Maximum Suppression (NMS):** The most critical step for eliminating redundant detections. NMS removes duplicates as follows:
  - Sort all remaining candidate boxes by descending confidence score ( $\Pr(\text{Class}_i | \text{Object}) \times \Pr(\text{Object})$ ).
  - Iteratively select the highest-confidence box  $B_{\max}$  as a final detection.
  - Compute the Intersection over Union (IoU) between  $B_{\max}$  and every other remaining box.
  - Suppress (discard) any box whose  $\text{IoU}(B_{\max}, B_i) \geq \text{threshold}$  (typically 0.45–0.6, class-dependent in multi-class settings).
  - Repeat until no boxes remain.

This greedy procedure ensures that each object is represented by exactly one high-confidence bounding box, significantly improving precision from the output.

## 2.3 Matrix Multiplication Free (MMF) Models

Matrix Multiplication Free (MMF) models, as introduced in [8], aim to eliminate the computationally expensive matrix multiplication (MatMul) operations that dominate the cost of modern deep neural networks. This subsection focuses on the core technique used in this project: replacing standard dense layers with MatMul-free dense layers based on ternary weights.

In a conventional dense (fully connected) layer, the output  $\mathbf{y} \in \mathbb{R}^m$  is computed through matrix multiplication as:

$$\mathbf{y} = \mathbf{x}\mathbf{W} = \sum_{j=1}^d x_j \mathbf{W}_{ij}, \quad i = 1, 2, \dots, m \quad (5)$$

where  $\mathbf{x} \in \mathbb{R}^d$  is the input vector and  $\mathbf{W} \in \mathbb{R}^{d \times m}$  is the weight matrix.

To remove MatMul operations, the approach is to replace standard dense layers with modules that use ternary weights constrained to the set  $\{-1, 0, +1\}$ . Let  $\tilde{\mathbf{W}}$  denote the ternary weight matrix. The operation then becomes a ternary accumulation:

$$\mathbf{y}_e = \mathbf{x} \circledast \tilde{\mathbf{W}} = \sum_{j=1}^d x_j \tilde{\mathbf{W}}_{ij}, \quad \tilde{\mathbf{W}}_{ij} \in \{-1, 0, +1\} \quad (6)$$

where  $\otimes$  represents the ternary matrix multiplication.

Because the weights are restricted to  $\{-1, 0, +1\}$ , the multiplication  $x_j \cdot \tilde{W}_{ij}$  simplifies to one of three cases:

$$x_j \tilde{W}_{ij} = \begin{cases} x_j, & \text{if } \tilde{W}_{ij} = +1 \\ 0, & \text{if } \tilde{W}_{ij} = 0 \\ -x_j, & \text{if } \tilde{W}_{ij} = -1 \end{cases} \quad (7)$$

Consequently, the entire operation reduces to simple additions and subtractions (accumulation):

$$y_{e,i} = \sum_{j:\tilde{W}_{ij}=1} x_j - \sum_{j:\tilde{W}_{ij}=-1} x_j, \quad i = 1, 2, \dots, m \quad (8)$$



## 3 Methods

### 3.1 Dataset

The experiments conducted in this project are performed using the CIFAR-10 dataset [9]. This is a widely used benchmark dataset in computer vision, consisting of small color images of size  $32 \times 32$  pixels in RGB format. The dataset contains 10 mutually exclusive classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck (see Figure 6).

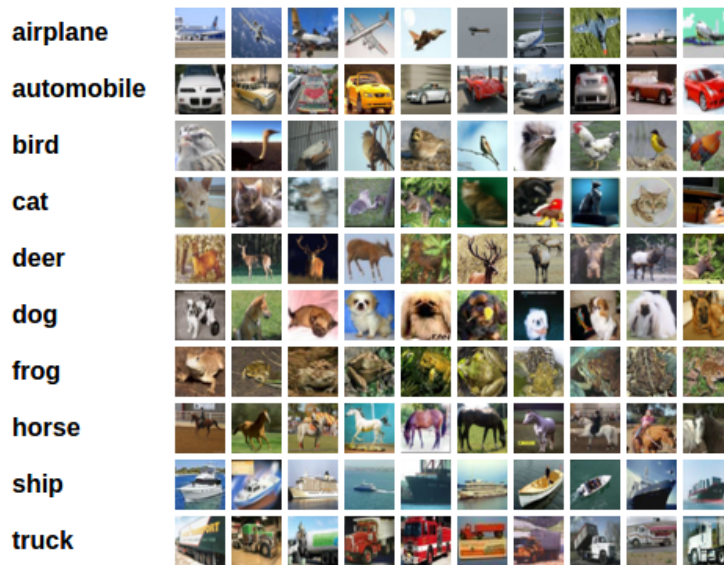


Figure 6: Sample images from CIFAR-10 dataset, showing one row per class.

It comprises a total of 60,000 images, with 6,000 images per class. The official split divides the data into 50,000 training images (5,000 per class) and 10,000 test images (1,000 per class). All images are of uniform resolution and each image contains a single dominant object belonging to one of the ten classes.

The CIFAR-10 dataset was chosen for this project for the next reasons:

- Small image size ( $32 \times 32$  pixels), allowing fast experimentation and lower computational requirements.
- Uniform resolution across all samples, eliminating the need for complex resizing or padding.
- Single-label images with one clearly distinguishable object per image, simplifying both training and evaluation.
- Sufficient number of samples per class (5,000 for training), providing a reasonable balance between dataset size and training speed.

These characteristics make CIFAR-10 suitable for comparing the performance of a standard MatMul-based model against a MMF model under controlled conditions.

### 3.1.1 Data Augmentation

Data augmentation is applied during training to artificially increase the diversity of the training set, reducing overfitting and improving generalization. All augmentations are applied on-the-fly at the image level before feeding samples to the network. No augmentation is applied at test time. The following transformations are employed:

#### Geometric transformations

Random affine transformations are applied to each image, combining translation of up to  $\pm 20\%$  of the image dimensions, scaling in the range  $[75\%, 125\%]$ , mild rotation up to  $\pm 15^\circ$ , and a shear of up to  $\pm 10^\circ$ . These perturbations simulate natural variation in object position, distance, and orientation within the scene.

#### Color jitter

Random perturbations are independently applied to brightness, contrast, and saturation, each sampled from a range of approximately  $[0.4\times, 1.6\times]$ , together with a small hue shift of up to  $\pm 0.08$ . This encourages the model to be invariant to moderate changes in lighting conditions and colour appearance.

#### Random horizontal flip

Each image is horizontally flipped with probability 0.5. This is a standard and highly effective augmentation for CIFAR-10, as most object classes are symmetric under reflection.

#### RandAugment

The RandAugment policy [14] is applied with two operations per image and a magnitude of 9. At each training step, two augmentation operations are randomly selected from a pre-defined pool (e.g., sharpness, posterize, equalize, rotate, solarize) and applied sequentially. This learned, search-free augmentation strategy has been shown to yield strong improvements on CIFAR-scale datasets.

#### Random erasing

A rectangular region of the image is randomly erased with probability 0.5, with the erased area covering between 2% and 40% of the total image area and an aspect ratio sampled from  $[0.3, 3.3]$ . The erased pixels are filled with random values. This technique, also known as Cutout [15], encourages the network to rely on distributed features rather than isolated regions.

#### Normalization

Finally, pixel values are normalized using the channel-wise mean  $(\mu_R, \mu_G, \mu_B) = (0.4914, 0.4822, 0.4465)$  and standard deviation  $(\sigma_R, \sigma_G, \sigma_B) = (0.2470, 0.2435, 0.2616)$ , computed from the CIFAR-10

training set. Normalization is always applied last, after all stochastic augmentations.

### Augmentation pipelines

Three augmentation pipelines of increasing strength are defined and used across experiments, summarized in Table 1.

The **Weak (W)** pipeline applies only basic geometric and color transformations without normalization, serving as the initial baseline configuration.

The **Strong (S)** pipeline extends this with horizontal flipping, wider color jitter ranges, and random erasing, and adds channel-wise normalization.

The **Strong+Norm (S+N)** pipeline further adds rotation, shear, RandAugment, and a higher random erasing probability, and is the most aggressive configuration explored in this project.

Table 1: Augmentation pipelines used across experiments. A dash indicates the transformation is not applied.

| Transformation         | W                         | S                               | S+N                              |
|------------------------|---------------------------|---------------------------------|----------------------------------|
| Rotation               | –                         | –                               | $\pm 15^\circ$                   |
| Translation            | $\pm 20\%$                | $\pm 20\%$                      | $\pm 20\%$                       |
| Scale                  | $\pm 20\%$                | $\pm 20\%$                      | $\pm 25\%$                       |
| Shear                  | –                         | –                               | $\pm 10^\circ$                   |
| Brightness             | $[0.67\times, 1.5\times]$ | $[0.5\times, 1.5\times]$        | $[0.4\times, 1.6\times]$         |
| Contrast               | –                         | $[0.7\times, 1.3\times]$        | $[0.5\times, 1.5\times]$         |
| Saturation             | $[0.67\times, 1.5\times]$ | $[0.5\times, 1.5\times]$        | $[0.4\times, 1.6\times]$         |
| Hue                    | –                         | $\pm 0.05$                      | $\pm 0.08$                       |
| Random horizontal flip | –                         | $p = 0.5$                       | $p = 0.5$                        |
| RandAugment [14]       | –                         | –                               | <code>num_ops= 2, mag.= 9</code> |
| Random erasing [15]    | –                         | $p = 0.4$ , scale $[2\%, 25\%]$ | $p = 0.5$ , scale $[2\%, 40\%]$  |
| Normalisation          | –                         | CIFAR-10                        | CIFAR-10                         |

## 3.2 Model Architectures and Implementation Details

Both the baseline and the Matrix Multiplication Free (MMF) models share an identical overall architecture, based on the first 20 convolutional layers of the original YOLOv1 network [11], adapted for image classification on CIFAR-10. The only difference between the two models lies in the implementation of the convolutional and linear layers: the baseline model uses standard matrix-multiplication-based layers, while the MMF model replaces them with custom MatMul-free equivalents.

### 3.2.1 Baseline Model Architecture

The baseline model (Standard Matrix Multiplication) consists of two main parts: a convolutional backbone (the first 20 layers from YOLOv1) and a lightweight classification head. The detailed layer configuration is presented in Table 2.

### 3.2.2 Baseline Model Implementation Details

All convolutional and linear layers use standard PyTorch implementations (`nn.Conv2d` and `nn.Linear`), which are designed to efficiently use the available tensor cores at the GPU.

Each tensor core in the GPU is responsible for performing a single matrix multiply-addition operation ( $D = A \times B + C$ ) at each clock cycle. The size of those matrices and the bit number of its values are determined by the hardware (GPU) model that is being used. For example, if the size of the matrices is  $4 \times 4$  and the supported bit format is bfloat16, a single tensor core is capable of computing 128 multiply-addition operations of 16 bit numbers at one clock cycle ( $4 \times 4 \times 4$  multiply +  $3 \times 4 \times 4$  addition +  $4 \times 4$  addition = 128 total).

In addition, the GPU has multiple tensor cores working on parallel, so that various matrix multiply-addition operations are performed in a single clock cycle.

### 3.2.3 MMF Model Architecture

The MMF model follows exactly the same architecture and layer configuration as the baseline model, as defined in Table 2. The only difference is the replacement of all standard convolutional and linear layers with custom MMF implementations: `MMFConv2d` instead of `nn.Conv2d`, and `MMFLinear` instead of `nn.Linear`. These custom layers implement ternary weights  $\{-1, 0, +1\}$  as described in Section 2.3.

| Part     | #  | Layer             | Input Ch | Output Ch | Kernel / Stride / Padding |
|----------|----|-------------------|----------|-----------|---------------------------|
| Backbone | 1  | Conv2d            | 3        | 64        | 7×7 / stride=2 / pad=3    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          |    | MaxPool2d         | -        | -         | 2×2 / stride=2            |
|          | 2  | Conv2d            | 64       | 192       | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          |    | MaxPool2d         | -        | -         | 2×2 / stride=2            |
|          | 3  | Conv2d            | 192      | 128       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 4  | Conv2d            | 128      | 256       | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 5  | Conv2d            | 256      | 256       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 6  | Conv2d            | 256      | 512       | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          |    | MaxPool2d         | -        | -         | 2×2 / stride=2            |
|          | 7  | Conv2d            | 512      | 256       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 8  | Conv2d            | 256      | 512       | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 9  | Conv2d            | 512      | 256       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 10 | Conv2d            | 256      | 512       | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 11 | Conv2d            | 512      | 256       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 12 | Conv2d            | 256      | 512       | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 13 | Conv2d            | 512      | 256       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 14 | Conv2d            | 256      | 512       | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 15 | Conv2d            | 512      | 512       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 16 | Conv2d            | 512      | 1024      | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          |    | MaxPool2d         | -        | -         | 2×2 / stride=2            |
|          | 17 | Conv2d            | 1024     | 512       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 18 | Conv2d            | 512      | 1024      | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 19 | Conv2d            | 1024     | 512       | 1×1 / stride=1 / pad=0    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
|          | 20 | Conv2d            | 512      | 1024      | 3×3 / stride=1 / pad=1    |
|          |    | LeakyReLU         | -        | -         | negative slope = 0.1      |
| Head     |    | AdaptiveAvgPool2d | -        | -         | output size = 1×1         |
|          |    | Flatten           | -        | -         | -                         |
|          |    | Linear            | 1024     | 10        | -                         |

Table 2: Detailed layer configuration of the model architecture (shared by both baseline and MMF models).

### 3.2.4 MMF Model Implementation Details on Dedicated Hardware

On a dedicated MMF accelerator (ASIC, FPGA), the weights are stored as 2-bit values. For each input  $x_j$ , the hardware computes equation 8 by performing the following steps in a single clock cycle (where only adders, a sign-control multiplexer, and a skip gate are used, without multiplier circuits):

- Read the 2-bit weight  $\tilde{W}_{ij}$ .
- If  $\tilde{W}_{ij} = 0$ : gate off the input (no addition/subtraction occurs).
- If  $\tilde{W}_{ij} = +1$ : feed  $x_j$  directly into the accumulator.
- If  $\tilde{W}_{ij} = -1$ : feed the negation of  $x_j$  ( $-x_j$ ) into the accumulator.

A multiplier circuit requires more logic gates than a simple adder/subtractor, which results in longer propagation delays and higher power consumption. This is why hardware-level MMF inference achieves latency and power reductions.

### 3.2.5 MMF Model Implementation Details on GPU

Implementing MMF (equation 8) in a single clock cycle is not possible at the GPU hardware level. For achieving this, the weight matrix  $\tilde{\mathbf{W}}$  would need to be looped value by value, checking if each  $\tilde{W}_{ij}$  is equal to  $-1$  or  $1$ , and performing the respective accumulation/subtraction ( $+x_j$  or  $-x_j$ ). This results in multiple single clock cycles to perform a single matrix multiply-addition operation, while the MatMul GPU implementation is capable of performing a single matrix multiply-addition operation per clock cycle.

This difference in the GPU hardware level benefits the matrix multiplication operations, providing them with higher number of operations per clock cycle. This results in much lower latency for the GPU implementation of MatMul, which means lower power consumption, even if the multiplier circuits are being used.

For this reason, it doesn't make sense to implement MMF at the GPU hardware level, as it would require long training times, high latency, and high power consumption. Instead:

- The hardware level MMF implementation is reserved for the dedicated accelerator, where low latency and low power consumption are achieved.
- The MMF model is trained in the GPU without the hardware level MMF implementation, by using the matrix multiply-addition operations of the tensor cores for faster training.

This means that the MMF model is "simulated" in the GPU, where the model is ensured to achieve a certain accuracy but the implementation on the hardware level is not really MMF, and later is deployed to dedicated hardware for real MMF hardware level implementation.

The "simulation" needs to take into account the restrictions that appear in the dedicated hardware, which are the limited bitwidth of the activations and the weight quantization. For this, [8] introduces an efficient GPU implementation for training the MMF layers, defining the forward and backward pass for training the MMF model on GPU, as defined in Algorithm 1.

The defined forward and backward passes in Algorithm 1 are the ones used for **MMFLinear**. However, for **MMFConv2d**, the algorithm needs to be adapted to the convolution, which is defined in Algorithm 2.

---

**Algorithm 1** Fused RMSNorm and BitLinear Algorithm with Quantization for **MMFLinear**

---

**Define ForwardPass**( $X, W, b, \epsilon$ )  
 $X \in \mathbb{R}^{M \times N}$ ,  $W \in \mathbb{R}^{N \times K}$ ,  $b \in \mathbb{R}^K$

**forward\_pass**( $X, W, b, \epsilon$ ):

Load  $X, W, b, \epsilon$  from HBM

**On Chip:**  $\tilde{Y}, \mu, \sigma^2, r \leftarrow \text{norm\_fwd}(X)$

**On Chip:**  $\tilde{W} \leftarrow \text{weight\_quant}(W)$

**On Chip:**  $O \leftarrow \tilde{Y} \circledast \tilde{W} + b$

Store  $O, \mu, \sigma^2, r$  to HBM

**Return**  $O, \mu, \sigma^2, r$

**norm\_fwd**( $X$ ):

$\mu, \sigma^2 \leftarrow \text{mean}(X), \text{variance}(X)$

$r \leftarrow 1/\sqrt{\sigma^2 + \epsilon}$

$\tilde{Y} \leftarrow \text{activation\_quant}(r(X - \mu))$

**Return**  $\tilde{Y}, \mu, \sigma^2, r$

**activation\_quant**( $X$ ):

$s \leftarrow 127/\max(|X|)$

$\tilde{X} \leftarrow \lfloor sX \rfloor_{[-128, 127]} \cdot (1/s)$

**Return**  $\tilde{X}$

**weight\_quant**( $W$ ):

$s \leftarrow 1/\text{mean}(|W|)$

$\tilde{W} \leftarrow \lfloor sW \rfloor_{[-1, 1]} \cdot (1/s)$

**Return**  $\tilde{W}$

$\lfloor X \rfloor_{[-a, a]}$ :

$X_r \leftarrow \text{round}(X)$

$X_c \leftarrow \text{clamp}(X_r)_{[-a, a]}$

**Return**  $X_c$

---

**Define BackwardPass**( $X, W, b, O, dO, \mu, \sigma^2, r$ )

$X \in \mathbb{R}^{M \times N}$ ,  $W \in \mathbb{R}^{N \times K}$ ,  $b \in \mathbb{R}^K$

$O \in \mathbb{R}^{M \times K}$ ,  $dO \in \mathbb{R}^{M \times K}$

**backward\_pass**( $X, W, b, O, \mu, \sigma^2, r, dO$ ):

Load  $X, W, b, O, \mu, \sigma^2, r, dO$  from HBM

**On Chip:**  $dY \leftarrow dO \times W^\top$

**On Chip:**  $dX, \tilde{Y} \leftarrow \text{norm\_bwd}(dY, X, \mu, \sigma^2, r)$

**On Chip:**  $dW \leftarrow dO^\top \times \tilde{Y}$

**On Chip:**  $db \leftarrow \text{sum}(dO)$

Store  $dX, dW, db$  to HBM

**Return**  $dX, dW, db$

**norm\_bwd**( $dY, X, \mu, \sigma^2, r$ ):

$\tilde{Y} \leftarrow \text{activation\_quant}(r(X - \mu))$

$d\sigma^2 \leftarrow \text{sum}(dY \odot (X - \mu) \odot (-0.5 \times r^3))$

$d\mu \leftarrow \text{sum}(-r dY) + d\sigma^2 \times \text{mean}(X - \mu)$

$dX \leftarrow r dY + 2d\sigma^2(X - \mu)/N + d\mu/N$

**Return**  $dX, \tilde{Y}$



---

**Algorithm 2** Fused RMSNorm and BitConv2d Algorithm with Quantization for MMFConv2d

---

**Define** ForwardPass( $X, W, b, \epsilon, s, p$ )

$X \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}}$

$W \in \mathbb{R}^{C_{out} \times C_{in} \times kH \times kW}, b \in \mathbb{R}^{C_{out}}$

$M = B \cdot H_{in} \cdot W_{in}$

**forward\_pass**( $X, W, b, \epsilon, s, p$ ):

Load  $X, W, b$  from HBM

$X_{\text{flat}} \leftarrow \text{reshape}(X) \in \mathbb{R}^{M \times C_{in}}$

**On Chip:**  $\tilde{Y}, \mu, \sigma^2, r \leftarrow \text{norm\_fwd}(X_{\text{flat}})$

**On Chip:**  $\tilde{W} \leftarrow \text{weight\_quant}(W)$

$\tilde{Y}_{4d} \leftarrow \text{reshape}(\tilde{Y}) \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}}$

**On Chip:**  $O \leftarrow \tilde{Y}_{4d} \otimes \tilde{W} + b$

Store  $O, \mu, \sigma^2, r$  to HBM

**Return**  $O, \mu, \sigma^2, r$

**norm\_fwd**( $X_{\text{flat}}$ ):

$\mu, \sigma^2 \leftarrow \text{mean}(X_{\text{flat}}), \text{variance}(X_{\text{flat}})$

$r \leftarrow 1/\sqrt{\sigma^2 + \epsilon}$

$\tilde{Y} \leftarrow \text{activation\_quant}(r(X_{\text{flat}} - \mu))$

**Return**  $\tilde{Y}, \mu, \sigma^2, r$

**activation\_quant**( $X$ ):

$s \leftarrow 127/\max(|X|)$

$\tilde{X} \leftarrow \lfloor sX \rfloor_{[-128, 127]} \cdot (1/s)$

**Return**  $\tilde{X}$

**weight\_quant**( $W$ ):

$s \leftarrow 1/\text{mean}(|W|)$

$\tilde{W} \leftarrow \lfloor sW \rfloor_{[-1, 1]} \cdot (1/s)$

**Return**  $\tilde{W}$

$\lfloor X \rfloor_{[-a, a]}$ :

$X_r \leftarrow \text{round}(X)$

$X_c \leftarrow \text{clamp}(X_r)_{[-a, a]}$

**Return**  $X_c$

---

**Define** BackwardPass( $X, W, b, dO, \mu, \sigma^2, r, s, p$ )

$X \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}}$

$W \in \mathbb{R}^{C_{out} \times C_{in} \times kH \times kW}, b \in \mathbb{R}^{C_{out}}$

$dO \in \mathbb{R}^{B \times C_{out} \times H_{out} \times W_{out}}$

$M = B \cdot H_{in} \cdot W_{in}$

**backward\_pass**( $X, W, b, \mu, \sigma^2, r, dO$ ):

Load  $X, W, b, \mu, \sigma^2, r, dO$  from HBM

$X_{\text{flat}} \leftarrow \text{reshape}(X) \in \mathbb{R}^{M \times C_{in}}$

**On Chip:**  $dY_{4d} \leftarrow dO \otimes^\top W$

$dY \leftarrow \text{reshape}(dY_{4d}) \in \mathbb{R}^{M \times C_{in}}$

**On Chip:**  $dX_{\text{flat}}, \tilde{Y} \leftarrow \text{norm\_bwd}(dY, X_{\text{flat}}, \mu, \sigma^2, r)$

$\tilde{Y}_{4d} \leftarrow \text{reshape}(\tilde{Y}) \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}}$

**On Chip:**  $dW \leftarrow dO \otimes^\top \tilde{Y}_{4d}$

**On Chip:**  $db \leftarrow \sum_{B, H_{out}, W_{out}} dO$

$dX \leftarrow \text{reshape}(dX_{\text{flat}}) \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}}$

Store  $dX, dW, db$  to HBM

**Return**  $dX, dW, db$

**norm\_bwd**( $dY, X_{\text{flat}}, \mu, \sigma^2, r$ ):

$\tilde{Y} \leftarrow \text{activation\_quant}(r(X_{\text{flat}} - \mu))$

$d\sigma^2 \leftarrow \sum (dY \odot (X_{\text{flat}} - \mu) \odot (-0.5 \cdot r^3))$

$d\mu \leftarrow \sum (-r \cdot dY) + d\sigma^2 \times \text{mean}(X_{\text{flat}} - \mu)$

$dX_{\text{flat}} \leftarrow r \cdot dY + 2 \cdot d\sigma^2 (X_{\text{flat}} - \mu) / C_{in} + d\mu / C_{in}$

**Return**  $dX_{\text{flat}}, \tilde{Y}$

### 3.3 Training Procedure

All models are trained using the same procedure to ensure fair comparison across experiments. The training pipeline is implemented in PyTorch and applied consistently to both the MatMul-based baseline and the MMF models.

#### Loss function

All models are trained as multi-class classifiers using cross-entropy loss, implemented via `nn.CrossEntropyLoss()`. This loss combines a log-softmax activation with the negative log-



likelihood criterion, and is the standard choice for single-label classification tasks such as CIFAR-10.

## Optimizer

Model parameters are updated using the AdamW optimizer [16], which extends the Adam optimizer by decoupling the weight decay from the gradient update. This decoupling has been shown to improve regularization compared to the  $L_2$  penalty used in standard Adam, making AdamW a widely adopted default for training deep networks. The key hyperparameters of the optimizer are the learning rate  $\eta$  and the weight decay coefficient  $\lambda$ .

## Learning rate schedulers

The learning rate schedule is treated as an experimental variable, and three strategies are evaluated across experiments:

- **CosineAnnealingLR** [17]: the learning rate is decayed from its initial value  $\eta$  to zero following a cosine curve over the total number of training epochs  $E$ .
- **CosineAnnealingLR with mid-training restart**: a variant of the above in which the learning rate is additionally halved at the midpoint of training (epoch  $E/2$ ), effectively applying two cosine decay phases of decreasing amplitude.
- **ReduceLROnPlateau**: the learning rate is reduced by a fixed factor when the validation accuracy metric stops improving for a given number of epochs (patience).

## Hyperparameters

The hyperparameters varied across experiments are the learning rate  $\eta$ , the weight decay  $\lambda$ , the batch size  $B$ , the number of training epochs  $E$ , and the learning rate scheduler. Unless otherwise stated, experiments follow a controlled protocol in which a single hyperparameter is varied while the remaining ones are held fixed at their default values, allowing their individual effect on model performance to be isolated. The specific values explored for each hyperparameter are reported in the corresponding experiment sections.

## 3.4 Evaluation Metrics

To assess and compare model performance across experiments, a set of metrics is tracked during training and evaluated at the end of each epoch. These metrics are recorded for both the training and validation splits unless otherwise stated, and cover three complementary aspects: predictive performance, computational cost, and resource usage.

### Loss

The cross-entropy loss is monitored on both the training and validation sets throughout training. The training loss indicates how well the model is fitting the training data, while the validation loss serves as the primary indicator of generalization and is used to detect overfitting.

## Accuracy

Top-1 classification accuracy is reported on both the training and validation sets. It is defined as the fraction of samples for which the class with the highest predicted probability matches the ground-truth label:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i] \quad (9)$$

where  $N$  is the number of samples,  $y_i$  is the ground-truth label,  $\hat{y}_i$  is the predicted label, and  $\mathbf{1}[\cdot]$  is the indicator function.

## Number of parameters

The total number of trainable parameters is reported for each model configuration, which allows comparison of the architectural complexity when the model architecture is modified.

## Memory usage

Peak GPU memory consumption (in GB) is recorded during training. This metric reflects the practical hardware requirements of each model. Is particularly relevant when modifying the architecture of the model, as well as when comparing MatMul-based training and the quantization aware training for MMF.

## Time per epoch

The wall-clock time required to complete one full training epoch is measured in seconds. This metric captures the computational efficiency of each model and training configuration.

# 3.5 Hardware and Software Environment

## Hardware

All experiments are conducted on a workstation equipped with two NVIDIA GeForce RTX 3090 Ti GPUs, each with 24 GB of VRAM, providing a total of 48 GB of GPU memory. Training is performed in parallel on both GPUs unless otherwise stated.

## Software

The implementation is written in Python using the PyTorch deep learning framework [18]. Development and experimentation are carried out in Visual Studio Code. The full source code for this project is publicly available in github <sup>2</sup>, which is explained in detail in Appendix A.

---

<sup>2</sup>[https://github.com/jonurce/MatMulFree\\_Image\\_Classification\\_Model](https://github.com/jonurce/MatMulFree_Image_Classification_Model)

## 4 Results and Discussion

### 4.1 Experiment Plan

This section presents the results of all experiments conducted throughout the project. The experimental strategy follows a controlled protocol in which one variable or architectural modification is investigated at a time, while the remaining configuration is held fixed. This allows the individual contribution of each design decision to be isolated and assessed independently. Experiments are organized into two main groups.

The first group focuses on the **Baseline Model**, covering hyperparameter tuning as well as an evaluation of its behavior when weights are ternarized at inference time.

The second group focuses on the **MMF Model**, and systematically explores a series of architectural and training modifications aimed at improving its performance.

Finally, the best configurations found in each group are directly compared in Section 4.4.

Table 3 summarizes all experiment groups, the variable or modification under investigation, and the number of runs performed in each case.

Table 3: Summary of all experiment groups conducted in this project.

| Model    | Experiment                  | Variable / Modification                            | Runs |
|----------|-----------------------------|----------------------------------------------------|------|
| Baseline | Hyperparameter Tuning       | $\eta$ , $\lambda$ , batch size, epochs, scheduler | 48   |
| Baseline | Ternary Inference           | Weight ternarization at inference                  | 1    |
| MMF      | Hyperparameter Tuning       | $\eta$ , $\lambda$ , batch size, epochs, scheduler | 35   |
| MMF      | Number of Channels          | Channel width                                      | 7    |
| MMF      | Weight initialization Scale | initialization scale factor                        | 8    |
| MMF      | Weight quantization Levels  | Ternary, quinary, septenary, nonary                | 7    |

The specific configuration used as the fixed baseline for each experiment group is reported at the beginning of the corresponding subsection. The complete results are documented in Appendix B.

### 4.2 Baseline Model

This subsection presents the experiments conducted on the Baseline Model. The first set of experiments focuses on hyperparameter tuning, exploring the effect of learning rate, weight decay, batch size, data augmentation, and learning rate scheduler on classification performance. The best configuration found is then used as the reference point for the next experiment, in which the trained Baseline Model is evaluated under MMF-style inference without retraining.

### 4.2.1 Hyperparameter Tuning

The goal of this experiment group is to find the best hyperparameter configuration for the Baseline Model on CIFAR-10. Starting from an arbitrary initial configuration, the learning rate  $\eta$ , weight decay  $\lambda$ , batch size  $B$ , data augmentation strategy, and learning rate scheduler are progressively tuned in a sequential, manual search. This section describes the key findings of each tuning phase and reports the most informative configurations in Table 4 and Figure 7.

Table 4: Selected runs from Baseline Model hyperparameter tuning. BS: batch size,  $\eta$ : learning rate,  $\lambda$ : weight decay, Aug: augmentation level (W: weak, S: strong, S+N: strong with normalization).

| Run | BS   | $\eta$             | $\lambda$          | Aug | Scheduler         | Train Acc | Val Acc |
|-----|------|--------------------|--------------------|-----|-------------------|-----------|---------|
| 0   | 2048 | $1 \times 10^{-3}$ | $6 \times 10^{-5}$ | W   | ReduceLROnPlateau | 44.0%     | 46.3%   |
| 1   | 1024 | $1 \times 10^{-3}$ | $6 \times 10^{-5}$ | W   | ReduceLROnPlateau | 87.4%     | 77.8%   |
| 3   | 1024 | $2 \times 10^{-3}$ | $6 \times 10^{-5}$ | W   | ReduceLROnPlateau | 88.9%     | 78.9%   |
| 7   | 1024 | $3 \times 10^{-3}$ | $6 \times 10^{-5}$ | W   | ReduceLROnPlateau | 10.0%     | 10.0%   |
| 17  | 1024 | $2 \times 10^{-3}$ | $6 \times 10^{-5}$ | S   | ReduceLROnPlateau | 77.7%     | 80.1%   |
| 24  | 1024 | $1 \times 10^{-3}$ | $6 \times 10^{-5}$ | S   | ReduceLROnPlateau | 84.0%     | 83.5%   |
| 27  | 1024 | $1 \times 10^{-3}$ | $6 \times 10^{-5}$ | S   | OneCycleLR        | 10.0%     | 10.0%   |
| 33  | 1024 | $8 \times 10^{-4}$ | $6 \times 10^{-5}$ | S   | ReduceLROnPlateau | 87.3%     | 84.0%   |
| 42  | 1024 | $8 \times 10^{-4}$ | $6 \times 10^{-5}$ | S+N | ReduceLROnPlateau | 73.4%     | 85.6%   |

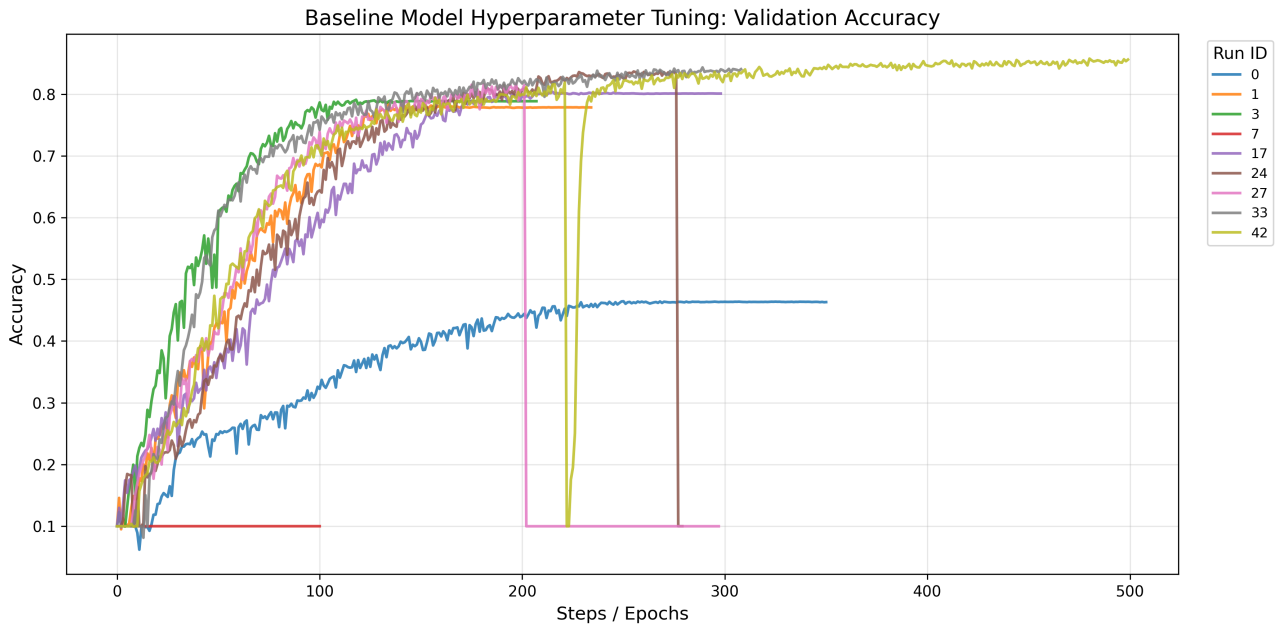


Figure 7: Validation Accuracy while Training for selected runs from Baseline Model hyperparameter tuning

### Phase 1. Batch size and learning rate (Runs 0–8)

A batch of 2048 with  $\eta = 10^{-3}$  yields validation accuracy of 46.3% (Run 0), while reducing the batch size to 1024 with the same  $\eta$  improves validation accuracy to 77.8% (Run 1). Increasing to  $\eta = 2 \times 10^{-3}$  further improves it to 78.9% (Run 3), but increasing it more or reducing it below  $10^{-3}$  degrades performance.

### Phase 2. Weight decay (Runs 9–16)

With the batch size fixed at 1024 and  $\eta = 2 \times 10^{-3}$ , the weight decay  $\lambda$  is explored around the default value of  $6 \times 10^{-5}$  used in Run 3. This phase reveals an extreme sensitivity to  $\lambda$ : values as close as  $5.9 \times 10^{-5}$  and  $6.1 \times 10^{-5}$  cause training failure, achieving 10% accuracy (random chance on a 10-class classification task).

### Phase 3. Data augmentation (Runs 17–24)

Applying the stronger augmentation from the pipeline (Section 3.1) to the best configuration from Phase 2 yields a notable improvement in validation accuracy, rising from 78.9% to 80.1% (Run 17), despite a reduction in training accuracy from 88.9% to 77.7%. Run 24, combines the strongest augmentation with a reduced learning rate of  $\eta = 10^{-3}$ , which achieves a new best validation accuracy of 83.5%.

### Phase 4. Learning rate scheduler (Runs 25–29)

A `ReduceLROnPlateau` scheduler with patience 15 and minimum learning rate  $\eta/20$  is used until this point. An attempt to replace this with a `OneCycleLR` scheduler results in unstable results, with some runs starting correctly and suddenly breaking, providing random change accuracy. All subsequent runs revert to `ReduceLROnPlateau`.

### Phase 5. Scheduler patience and learning rate refinement (Runs 30–35)

Increasing the scheduler patience to 25 and reducing  $\eta$  to  $6 \times 10^{-4}$  yields a slight improvement in training accuracy but not in validation (Run 32). Increasing  $\eta$  to  $8 \times 10^{-4}$  achieves a new best validation accuracy of 84.0% (Run 33), which is not improved by further adjustments to patience or learning rate (Runs 34–35).

### Phase 6. Strongest augmentation with normalization (Runs 36–46)

The strongest augmentation pipeline, now with normalization applied inside the transform (Section 3.1), is evaluated in Run 42 using the best configuration from Run 33 ( $\eta = 8 \times 10^{-4}$ ,  $\lambda = 6 \times 10^{-5}$ , batch size 1024). Despite a lower training accuracy of 73.4%, this run achieves the best validation accuracy of the entire search at **85.6%** (Run 42).

Attempts to use lower or higher  $\eta$  with this augmentation (Runs 37–41) result in training failures. Varying the `ReduceLROnPlateau` reduction factor around 0.5 (Runs 44–46) does not improve upon Run 42. Run 42 is therefore selected as the final Baseline Model configuration (Table 5).

## Best configuration

Therefore, the best Baseline Model configuration is Run 42, which is used as reference for all subsequent experiments. This configuration is summarized in Table 5.

Table 5: Best Baseline Model configuration (Run 42).

| Hyperparameter         | Value                     |
|------------------------|---------------------------|
| Batch size             | 1024                      |
| Learning rate $\eta$   | $8 \times 10^{-4}$        |
| Weight decay $\lambda$ | $6 \times 10^{-5}$        |
| Optimiser              | AdamW                     |
| Scheduler              | ReduceLROnPlateau         |
| Scheduler parameters   | Factor 0.5, patience 25   |
| Data augmentation      | Strongest + Normalization |
| Max epochs             | 2000                      |
| <b>Train accuracy</b>  | <b>73.4%</b>              |
| <b>Val accuracy</b>    | <b>85.6%</b>              |

### 4.2.2 Baseline Model with Ternary Inference

The goal of this experiment is to assess whether the best Baseline Model can be directly converted to MMF-style inference without any retraining. Specifically, the trained Baseline Model weights are kept fixed, and the standard forward pass is replaced by the MMF forward pass, which quantizes activations to 8-bit integers and ternarizes the weight matrices (as explained in Section 2.3). If successful, this would allow a standard MatMul-based model to be directly mapped as a MMF model, without any specific training for the MMF model.

The results are reported in Table 6. The original Baseline Model achieves a validation accuracy of 85.6% with a validation loss of 0.461, consistent with the results obtained during training (Table 5). However, applying the MMF forward pass at inference causes a complete collapse in performance: both training and validation accuracy drop to 10.0%, which corresponds to random chance on a 10-class classification task, and the loss increases to approximately 3.388 in both cases.

Table 6: Comparison of the Baseline Model under standard and MMF-style inference.

| Inference mode | Train Loss | Train Acc | Val Loss | Val Acc |
|----------------|------------|-----------|----------|---------|
| Standard       | 0.758      | 73.4%     | 0.461    | 85.6%   |
| MMF (Ternary)  | 3.388      | 10.00%    | 3.388    | 10.00%  |

## 4.3 MMF Model

This subsection presents the experiments conducted on the MMF Model. Experiments are organized as a sequential refinement process, each subsection introducing a targeted architectural or training modification, building on the best configuration found in the previous step.

### 4.3.1 Hyperparameter Tuning

#### Motivation

Before introducing any architectural modifications, the MMF Model’s hyperparameters are tuned. Given that the MMF forward pass introduces quantization and ternarization at every layer, the training dynamics are expected to differ, and a dedicated tuning phase is therefore necessary. The goal is to find a good training configuration for the MMF Model. In addition, the raw effects of changing the model to MMF can be observed.

#### Setup

The MMF Model used in this subsection shares the exact same architecture as the Baseline, but using MMF layers (as explained in section 3.2). The hyperparameters explored are the learning rate  $\eta$ , the weight decay  $\lambda$ , the maximum number of training epochs  $E$ , and the learning rate scheduler. The data augmentation pipeline is fixed to S+N, following the conclusion of the Baseline tuning.

#### Results

Table 7: Selected runs from MMF Model hyperparameter tuning. BS: batch size,  $\eta$ : learning rate,  $\lambda$ : weight decay.

| Run       | BS          | $\eta$                               | $\lambda$ | Epochs     | Scheduler                                | Train Acc    | Val Acc      |
|-----------|-------------|--------------------------------------|-----------|------------|------------------------------------------|--------------|--------------|
| 9         | 1024        | $4 \times 10^{-3}$                   | 0         | 1000       | Cosine                                   | 40.9%        | 54.8%        |
| 10        | 1024        | $2 \times 10^{-3}$                   | 0         | 1000       | Cosine                                   | 39.8%        | 51.8%        |
| 17        | 1024        | $3 \times 10^{-3}$                   | 0         | 500        | Cosine (min_lr = $0.1\eta$ )             | 42.9%        | 57.3%        |
| 18        | 1024        | $3 \times 10^{-3}$                   | 0         | 500        | Cosine + half at $T/2$                   | 40.9%        | 54.5%        |
| 19        | 1024        | $4 \times 10^{-3}$                   | 0         | 500        | Cosine + half at $T/2$                   | 43.5%        | 57.1%        |
| <b>20</b> | <b>1024</b> | <b><math>3 \times 10^{-3}</math></b> | <b>0</b>  | <b>800</b> | <b>Cosine + half at <math>T/2</math></b> | <b>47.1%</b> | <b>60.2%</b> |
| 34        | 1024        | $3 \times 10^{-3}$                   | 0         | 1200       | Cosine + half at $T/3$ + flat            | 47.4%        | 60.3%        |

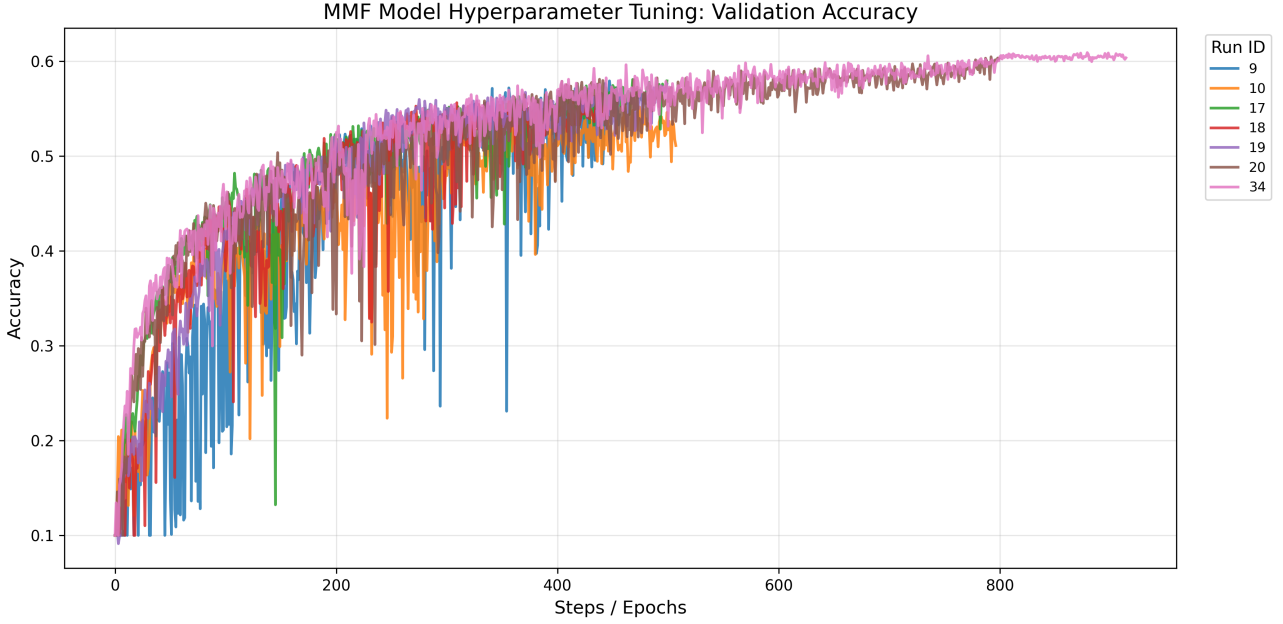


Figure 8: Validation Accuracy while Training for chosen Hyperparameter Tuning Experiments with MMF Model

Early runs (Runs 0–7) establish that the `CosineAnnealingLR` scheduler works best for the MMF Model and a relatively high learning rate in the range  $[3 \times 10^{-3}, 4 \times 10^{-3}]$  with zero weight decay, achieving a validation accuracy of approximately 55%.

Motivated by the original MMF paper [8], the cosine decay scheduler halves the learning rate at half of the total epochs. With  $\eta = 3 \times 10^{-3}$  and 800 epochs, this achieves a new best validation accuracy of approximately 60% in Run 20, with the model still appearing to converge at the end of training.

Runs 12–25 attempt to extend this further by training for longer (up to 1200 epochs), appending a flat low learning rate phase after the cosine decay, or chaining multiple cosine cycles. However, none of these strategies improve upon Run 20: extended flat phases cause the training to stagnate, chained cosine cycles trigger early stopping due to unstable learning rate transitions, and higher learning rates ( $4 \times 10^{-3}$ ) produce erratic loss spikes in the later stages of training. The key results are summarized in Table 7.

## Discussion

The best MMF configuration (Run 20) achieves a validation accuracy of 60.2%, compared to 85.6% for the best Baseline Model. This large gap, despite both models sharing the same architectural structure and training data, is expected and can be attributed to the fundamental representational constraint imposed by the MMF layers. Whereas the Baseline Model uses full-precision `float32` weights, which can take any of  $2^{32}$  possible values, the MMF Model restricts weights to only three discrete values  $\{-1/s_w, 0, +1/s_w\}$ , where  $s_w = 1/\text{mean}(|W|)$ .

A natural way to partially compensate for this reduced per-weight expressiveness is to increase



the number of channels in each layer, providing the model with more representational degrees of freedom. This is the modification explored in the following subsection.

### 4.3.2 Modifying Number of Channels

#### Motivation

As discussed in the previous subsection, the accuracy gap between the MMF and Baseline models is primarily attributed to the severely reduced representational capacity of ternary weights. A natural way to partially compensate for this is to increase the number of channels in each layer, providing the model with more filters and therefore more representational degrees of freedom. Conversely, reducing the number of channels may confirm that reducing the network’s size results in worse results.

#### Setup

All runs use the same training configuration as MMF Run 20: batch size 1024,  $\eta = 3 \times 10^{-3}$ ,  $\lambda = 0$ , and the cosine scheduler with the learning rate halved at  $T/2$ .

The channel scaling factor  $\alpha$  is the only variable modified across runs, scaling all channel widths in `MMFConv2d` and `MMFLinear` layers by a factor  $\alpha \in \{0.5, 0.75, 1.5, 2, 3\}$ .

The number of trainable parameters, GPU memory consumption (for each of the 2 GPUs used for training in parallel), and training time per epoch are recorded alongside accuracy to assess the computational cost of each configuration.

#### Results and Discussion

Table 8: Results for channel scaling experiments.

| Run | $\alpha$ | LR                 | Params | GB/GPU | Time/epoch | Train Acc | Val Acc |
|-----|----------|--------------------|--------|--------|------------|-----------|---------|
| 0   | 0.5      | $3 \times 10^{-3}$ | 5.6M   | 3.7    | 7s         | 39.0%     | 52.7%   |
| 1   | 0.5      | $2 \times 10^{-3}$ | 5.6M   | 3.7    | 7s         | 40.0%     | 52.6%   |
| 2   | 0.75     | $3 \times 10^{-3}$ | 12.6M  | 5.3    | 12s        | 40.3%     | 53.1%   |
| 3   | 1.0      | $3 \times 10^{-3}$ | 22.4M  | 7.0    | 20s        | 47.1%     | 60.2%   |
| 4   | 1.5      | $4 \times 10^{-3}$ | 50.4M  | 10.5   | 36s        | 47.2%     | 61.3%   |
| 5   | 2.0      | $3 \times 10^{-3}$ | 89.6M  | 13.0   | 60s        | 50.5%     | 63.4%   |
| 6   | 3.0      | $3 \times 10^{-3}$ | 201.5M | 21.6   | 110s       | 49.7%     | 63.8%   |

The results suggest that scaling the number of channels for the MMF Model improves its accuracy, at the cost of increasing the parameter count, GPU memory and training time per epoch. However, for future experiments in the next subsections, the original channel configuration ( $\alpha = 1.0$ , MMF Run 20) is retained, as its accuracy-to-cost ratio is the most favorable and allows faster training.

### 4.3.3 Modifying Weight Initialization Scale

#### Motivation

The ternarization function in the forward pass maps the weights  $W$  as  $W/\text{mean}(|W|)$ , then rounds the values, and clamps between -1 and 1 (as described in Algorithm 1 and 2). Until this point, weights of the MMF Model are randomly initialized following a normal distribution of values between -1 and 1. Increasing/decreasing the weight initialization scale leads to higher/lower  $W$  values and a higher/lower  $\text{mean}(|W|)$ , which means that  $W/\text{mean}(|W|)$  is affected before doing the round and clamp for ternarization, resulting in a different initial distribution of ternary weights (higher or lower weight values falling to zero). This subsection investigates the effect of modifying the weight initialization scale, without any change in model size or architecture.

#### Setup

All runs use the same training configuration as MMF Run 20: batch size 1024,  $\eta = 3 \times 10^{-3}$ ,  $\lambda = 0$ , and the cosine scheduler with the learning rate halved at  $T/2$ .

The weight initialization scale is the only variable modified across runs, applied as a multiplicative factor to the standard normal initialization. A total of 6 runs are conducted, exploring scales in  $\{0.1, 0.2, 0.5, 1.0, 10\}$ .

#### Results and Discussion

Table 9: Results for weight initialization scale experiments.  $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization.

| Run      | Scale      | LR                                   | $\text{mean}( W )$ | $W^-$        | $W^0$        | $W^+$       | Train Acc    | Val Acc      |
|----------|------------|--------------------------------------|--------------------|--------------|--------------|-------------|--------------|--------------|
| 0        | 10.0       | $3 \times 10^{-3}$                   | 7.975              | 34.5%        | 31.0%        | 34.5%       | 14.7%        | 17.6%        |
| 1        | 1.0        | $3 \times 10^{-3}$                   | 0.839              | 34.4%        | 31.1%        | 34.5%       | 47.1%        | 60.2%        |
| 2        | 0.5        | $3 \times 10^{-3}$                   | 0.455              | 34.3%        | 31.2%        | 34.3%       | 51.3%        | 64.6%        |
| <b>3</b> | <b>0.2</b> | <b><math>3 \times 10^{-3}</math></b> | <b>0.247</b>       | <b>34.3%</b> | <b>31.3%</b> | <b>34.3</b> | <b>53.0%</b> | <b>66.3%</b> |
| 4        | 0.1        | $3 \times 10^{-3}$                   | 0.079              | 33.9%        | 31.2%        | 34.8%       | 10.0%        | 10.0%        |
| 5        | 0.1        | $2 \times 10^{-3}$                   | 0.079              | 34.1%        | 31.1%        | 34.7%       | 10.0%        | 10.0%        |
| 6        | 0.1        | $8 \times 10^{-4}$                   | 0.099              | 34.3%        | 31.2%        | 34.4%       | 52.6%        | 66.1%        |
| 7        | 0.1        | $6 \times 10^{-4}$                   | 0.092              | 34.3%        | 31.2%        | 34.4%       | 50.3%        | 63.2%        |

Results reveal a narrow optimal range for the weight initialization scale, with values of 0.2–0.5 consistently outperforming the default scale of 1.0. As correctly predicted, a smaller scale reduces the magnitude of  $\text{mean}(|W|)$ . However, the ternarized weight distribution in positive/zero/negative values doesn’t change too much among the runs. From these results, it can be observed that under similar weight distributions (negative/zero/positive), smaller values of  $\text{mean}(|W|)$  provide better accuracy, but with a limit.

#### 4.3.4 Ternary, Quinary, Septenary, Nonary

##### Motivation

The original MMF Model restricts weights to three discrete values  $\{-1/s_w, 0, +1/s_w\}$ , where  $s_w = 1/\text{mean}(|W|)$ . While this ternary scheme offers significant computational advantages, it severely limits the representational capacity of each weight. A natural extension is to allow more discrete levels, moving from ternary quantization ( $\{-1, 0, +1\}$ ) to quinary ( $\{-2, -1, 0, +1, +2\}$ ), septenary ( $\{-3, -2, -1, 0, +1, +2, +3\}$ ), or nonary ( $\{-4, \dots, +4\}$ ), all scaled by  $1/s_w$ . This increases representational capacity while preserving the integer-valued structure of the weights. Formally, the weight quantization is generalized as:

$$\tilde{W} = \lfloor s_w \cdot W \rfloor_{[-k,k]} \cdot \frac{1}{s_w}, \quad s_w = \frac{k}{\text{mean}(|W|)} \quad (10)$$

where  $k \in \{1, 2, 3, 4\}$  controls the number of quantization levels ( $2k + 1 \in \{3, 5, 7, 9\}$ ). This subsection investigates whether increasing the number of discrete weight levels improves classification accuracy over the ternary baseline.

##### Setup

All runs use the best configuration from the weight initialization experiments (previous subsection Run 3): batch size 1024,  $\eta = 3 \times 10^{-3}$ ,  $\lambda = 0$ , weight initialization scale 0.2, and the cosine scheduler with the learning rate halved at  $T/2$ .

The quantization level  $k$  and the weight initialization scale are the only variables modified across runs. A total of 6 runs are conducted. The weight distribution after ternarization is monitored via  $\text{mean}(|W|)$  and the proportion of negative, zero, and positive weights, to verify that the quantization behaves as expected across all levels.

##### Results and Discussion

Table 10: Results for quantization level experiments. W Init: weight initialization scale.  $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization.

| Run      | k        | Levels   | $s_w$                                  | W Init     | $\text{mean}( W )$ | $W^-$        | $W^0$        | $W^+$        | Train Acc    | Val Acc      |
|----------|----------|----------|----------------------------------------|------------|--------------------|--------------|--------------|--------------|--------------|--------------|
| 0        | 1        | 3        | $k/\text{mean}( W )$                   | 0.2        | 0.2475             | 34.3%        | 31.3%        | 34.3%        | 53.0%        | 66.3%        |
| <b>1</b> | <b>2</b> | <b>5</b> | <b><math>1/\text{mean}( W )</math></b> | <b>0.2</b> | <b>0.2624</b>      | <b>34.2%</b> | <b>31.6%</b> | <b>34.2%</b> | <b>53.8%</b> | <b>67.1%</b> |
| 2        | 2        | 5        | $k/\text{mean}( W )$                   | 0.2        | 0.1630             | 34.0%        | 31.2%        | 34.8%        | 10.0%        | 10.0%        |
| 3        | 2        | 5        | $k/\text{mean}( W )$                   | 0.5        | 0.2624             | 34.2%        | 31.6%        | 34.2%        | 53.5%        | 65.9%        |
| 4        | 3        | 7        | $k/\text{mean}( W )$                   | 0.5        | 0.4326             | 34.3%        | 31.2%        | 34.5%        | 54.2%        | 67.0%        |
| 5        | 3        | 7        | $1/\text{mean}( W )$                   | 0.5        | 0.4644             | 34.3%        | 31.1%        | 34.6%        | 45.5%        | 59.4%        |
| 6        | 4        | 9        | $k/\text{mean}( W )$                   | 0.5        | 0.4298             | 34.3%        | 31.2%        | 34.5%        | 54.2%        | 66.4%        |

Increasing the number of discrete weight levels beyond ternary yields small improvements in validation accuracy, with the best result of 67.1% (quinary,  $k = 2$ ) exceeding the ternary reference by less than 1 percentage point. The weight distributions across all successful runs remain near-identical, with approximately 31% of weights at zero regardless of  $k$ . The additional discrete levels do not significantly enrich the learned representations.

## 4.4 Best Model Comparison

This subsection compares the best configurations obtained for the Baseline and MMF models across the experiments described in the previous subsections. The goal is to consolidate the key findings and quantify the performance gap between full-precision and MatMul-free architectures under optimal training conditions for each.

### 4.4.1 Configuration Summary

Table 11 summarizes the best training configurations for both models, the best baseline model being Run 42 from Table 4, and the best MMF Model being Run 1 from Table 10.

Table 11: Best training configurations for Baseline and MMF models.

| Hyperparameter         | Baseline Model     | MMF Model                       |
|------------------------|--------------------|---------------------------------|
| Architecture           | Standard Conv2d    | MMFConv2d + MMFLinear           |
| Batch size             | 1024               | 1024                            |
| Learning rate $\eta$   | $8 \times 10^{-4}$ | $3 \times 10^{-3}$              |
| Weight decay $\lambda$ | $6 \times 10^{-5}$ | 0                               |
| Optimiser              | AdamW              | AdamW                           |
| Scheduler              | ReduceLROnPlateau  | CosineAnnealing + half at $T/2$ |
| Data augmentation      | Strongest (S+N)    | Strongest (S+N)                 |
| Max epochs             | 2000               | 800                             |
| Weight quantization    | Float32            | Quinary ( $k = 2$ )             |
| Weight init scale      | PyTorch default    | 0.2                             |
| Parameters             | 22.04M             | 22.4M                           |

### 4.4.2 Training Comparison

While both models share the same architectural structure (identical layer distribution), they differ substantially in their optimal hyperparameters and training dynamics.

As shown in Figure 9, the Baseline Model requires a lower learning rate ( $8 \times 10^{-4}$ ) while the MMF Model performs best with a higher learning rate ( $3 \times 10^{-3}$ ). In addition, the Baseline converges well with `ReduceLROnPlateau`, whereas the MMF Model requires a structured two-phase cosine schedule.

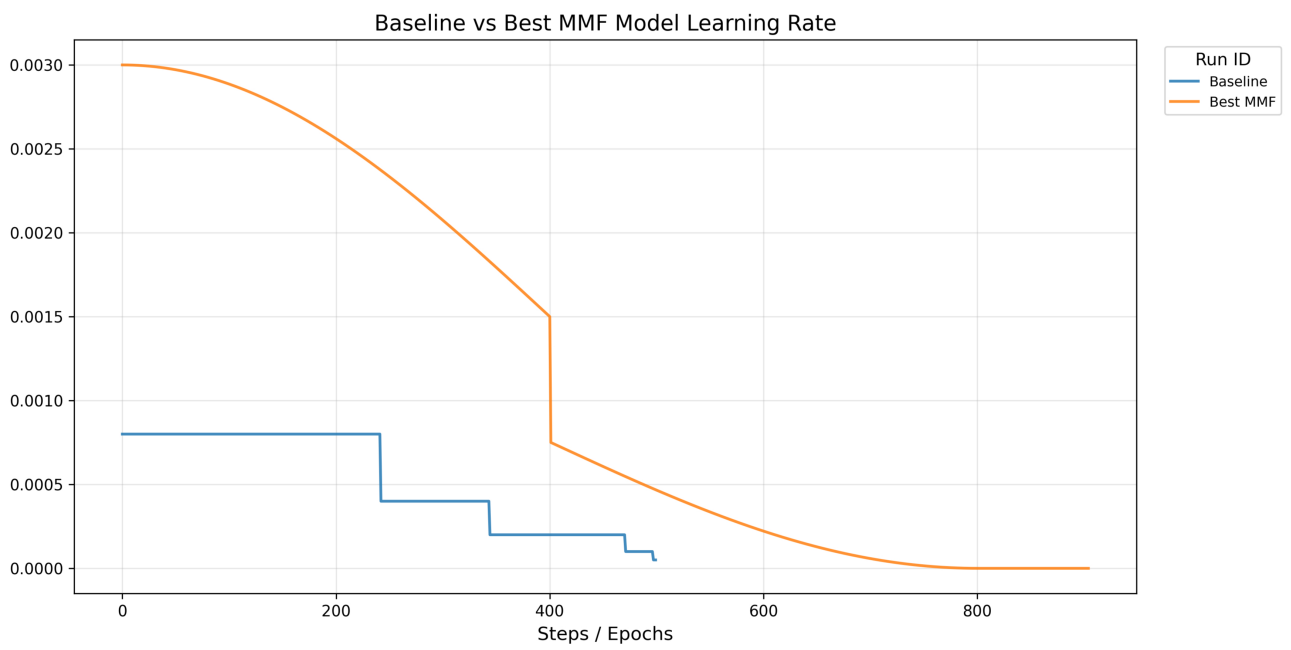


Figure 9: Learning Rate while Training for Baseline vs Best MMF Model

Figure 10 and 11 show respectively how the training accuracy and the validation accuracy converge while training. In both cases, the training accuracy is lower than the validation accuracy. In addition, there is a noticeable accuracy drop spike (at around epoch 220) while training the baseline model, which is then recovered as the training process continues.

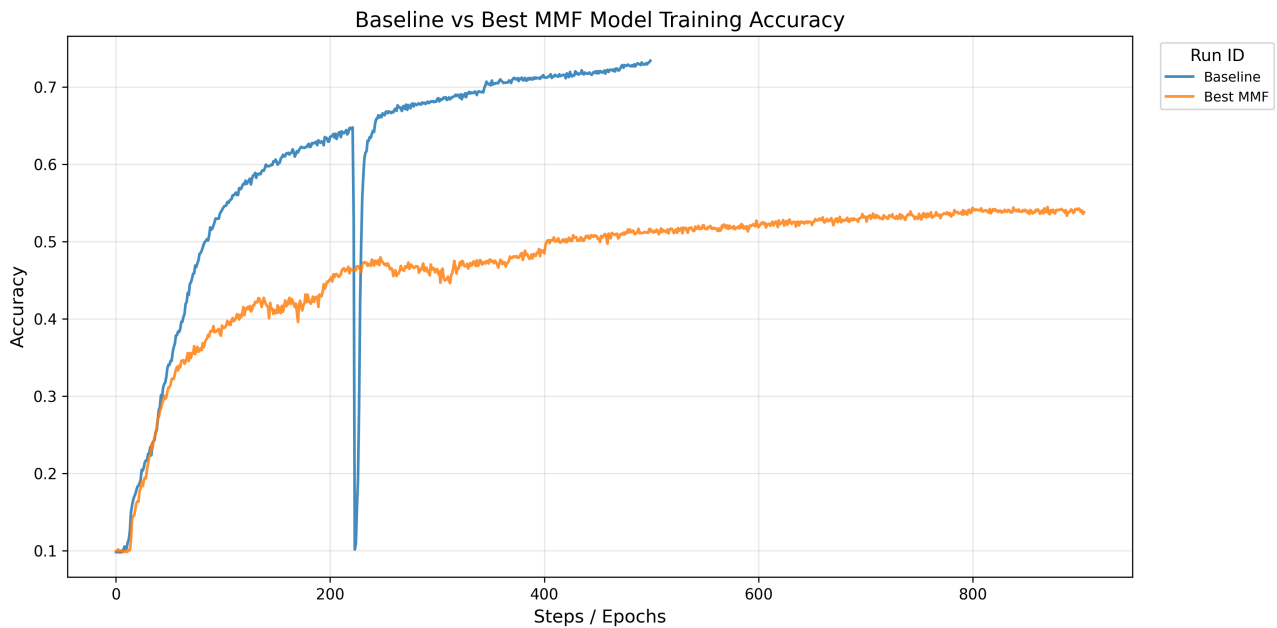


Figure 10: Training Accuracy while Training for Baseline vs Best MMF Model

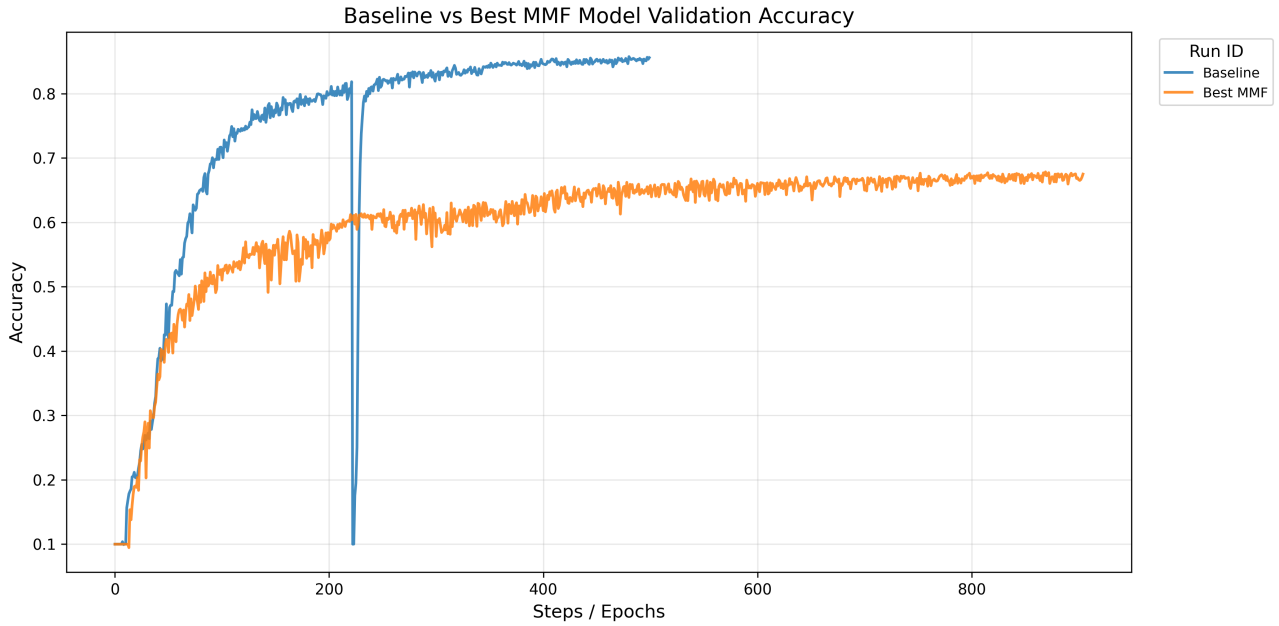


Figure 11: Validation Accuracy while Training for Baseline vs Best MMF Model

#### 4.4.3 Overall Performance Comparison

Table 12 presents the final classification accuracy for both models on the CIFAR-10 dataset. The Baseline Model achieves a validation accuracy of 85.6%, while the best MMF Model reaches 67.1%, resulting in a performance gap of 18.5 percentage points. Both models exhibit a similar gap between training and validation accuracy of approximately 12–14 percentage points.

Table 12: Performance comparison between best Baseline and MMF models.

| Model                 | Train Acc. | Validation Acc. | Train-Val Gap |
|-----------------------|------------|-----------------|---------------|
| Baseline              | 73.4%      | <b>85.6%</b>    | -12.2 pp      |
| Best MMF              | 53.8%      | <b>67.1%</b>    | -13.3 pp      |
| <b>Baseline - MMF</b> | +19.6 pp   | <b>+18.5 pp</b> | +1.1 pp       |

In the other hand, Table 13 shows the Top-1, Top-3, and Top-5 accuracies of both models. The gap between both models decreases as the Top-K increases.

Table 13: Top-K Validation Accuracy Comparison on CIFAR-10 Validation Set.

| Model                 | Top-1     | Top-3    | Top-5    |
|-----------------------|-----------|----------|----------|
| Baseline Model        | 85.53%    | 97.29%   | 99.26%   |
| Best MMF Model        | 67.72%    | 90.39%   | 96.46%   |
| <b>Baseline - MMF</b> | +17.81 pp | +6.90 pp | +2.80 pp |

Finally, figure 12 displays the average confidence value of the correct and incorrect answer for both models. The Best MMF Model has a lower average in both correct and incorrect predictions, showing that is less greedy when predicting a class.

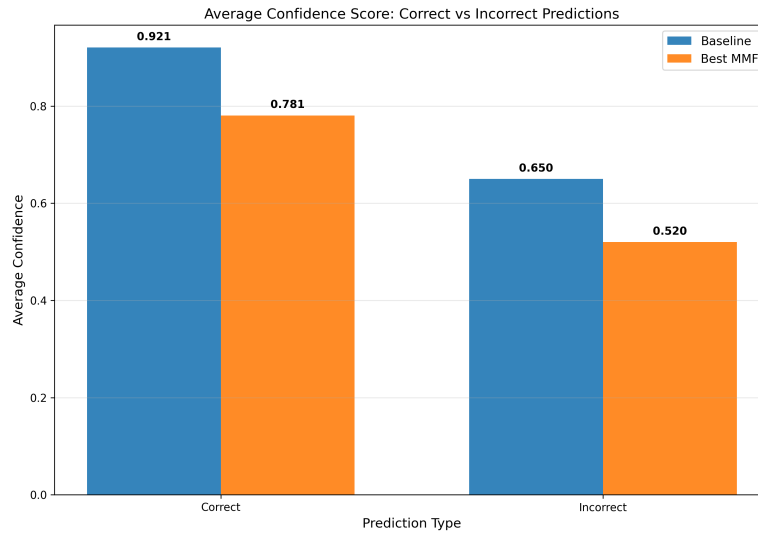


Figure 12: Average Confidence Score for Correct and Incorrect Predictions on Validation Data for Baseline vs Best MMF Model

#### 4.4.4 Per Class Performance Comparison

In the one hand, the confusion matrices for the Baseline Model and the Best MMF Model are displayed respectively in Figures 13 and 14. Both models have a clear diagonal, the Baseline Model being stronger than the MMF Model. However, the Baseline Model still has some confusion, specially with the dog and cat classes. This confusion between dogs and cats is bigger in the Best MMF Model, where additional confusions, such as the one between truck-automobile and airplane-ship, become more relevant.

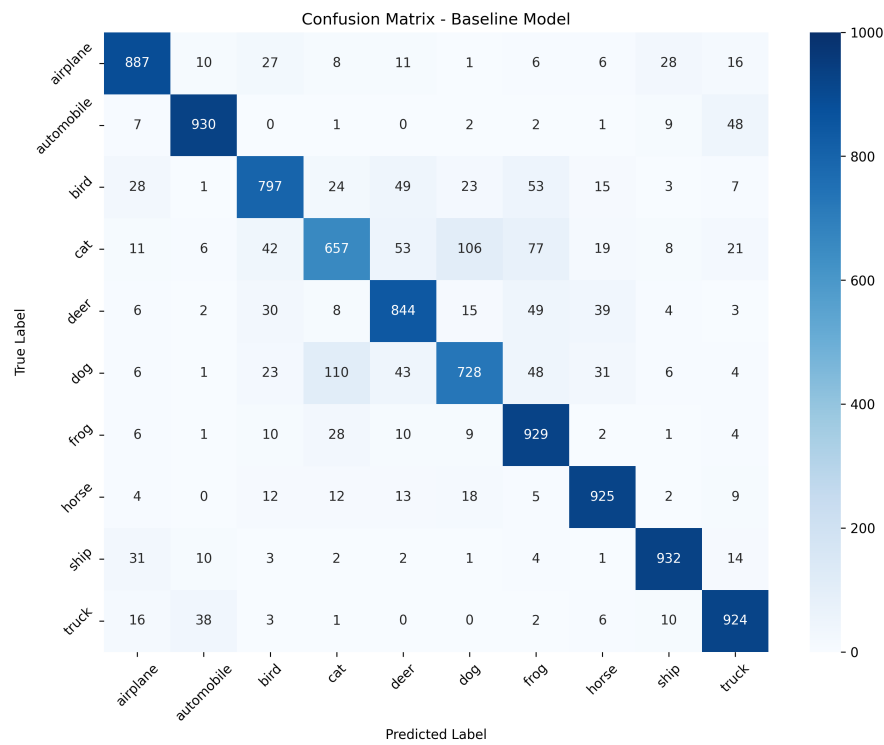


Figure 13: Confusion Matrices for Baseline Model

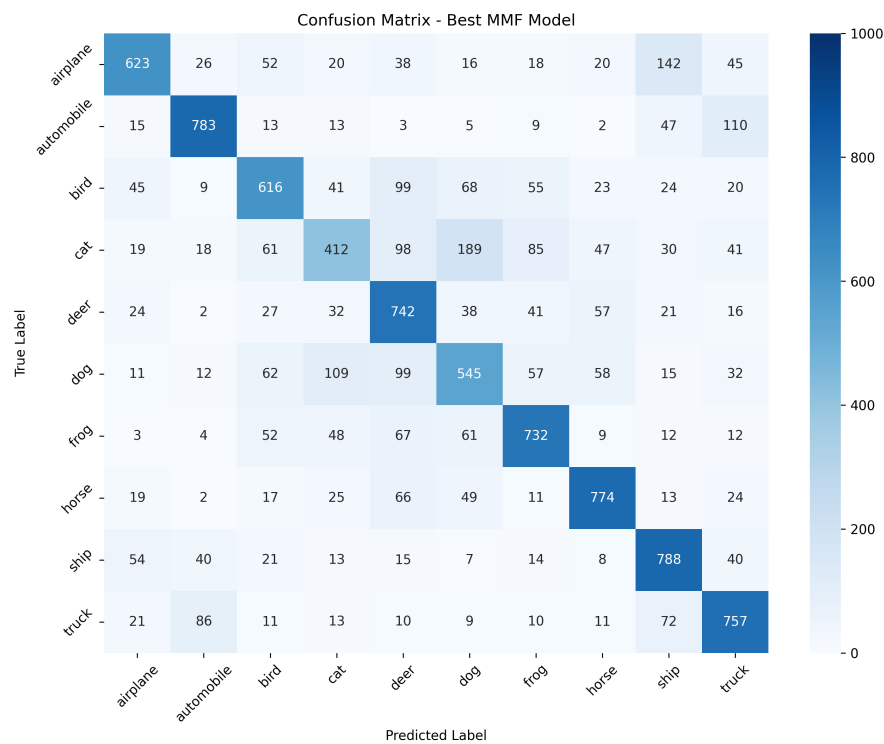


Figure 14: Confusion Matrices for Best MMF Model



In the other hand, the Top-1, Top-3, and Top-5 validation accuracies are measured per class in Figures 15, 16, and 17. It can be observed that both models follow similar patterns, predicting cats and dogs with the lower accuracy and ships with the highest.

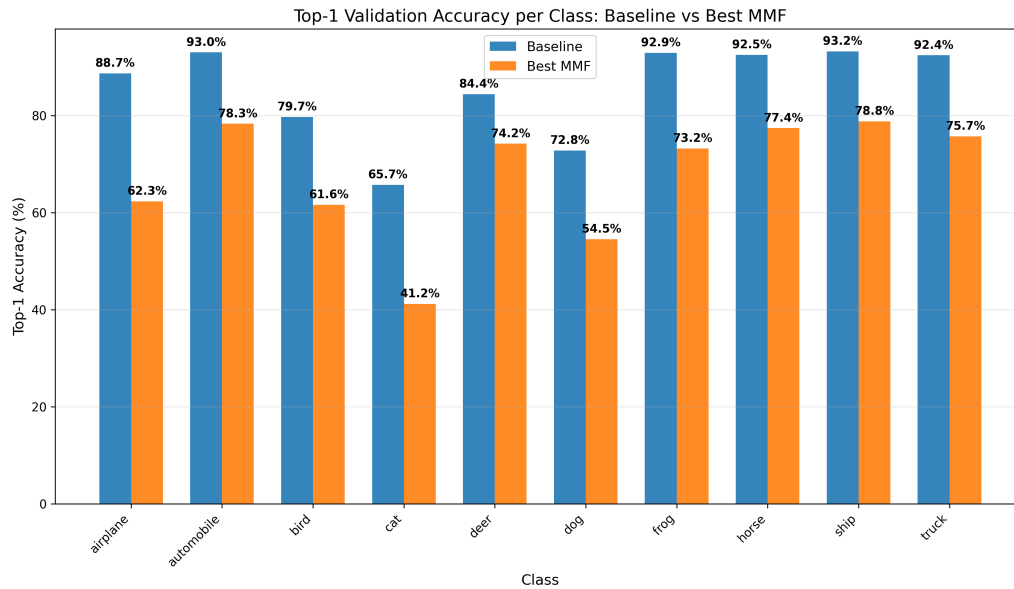


Figure 15: Baseline vs Best MMF Model Per Class Top-1 Accuracy

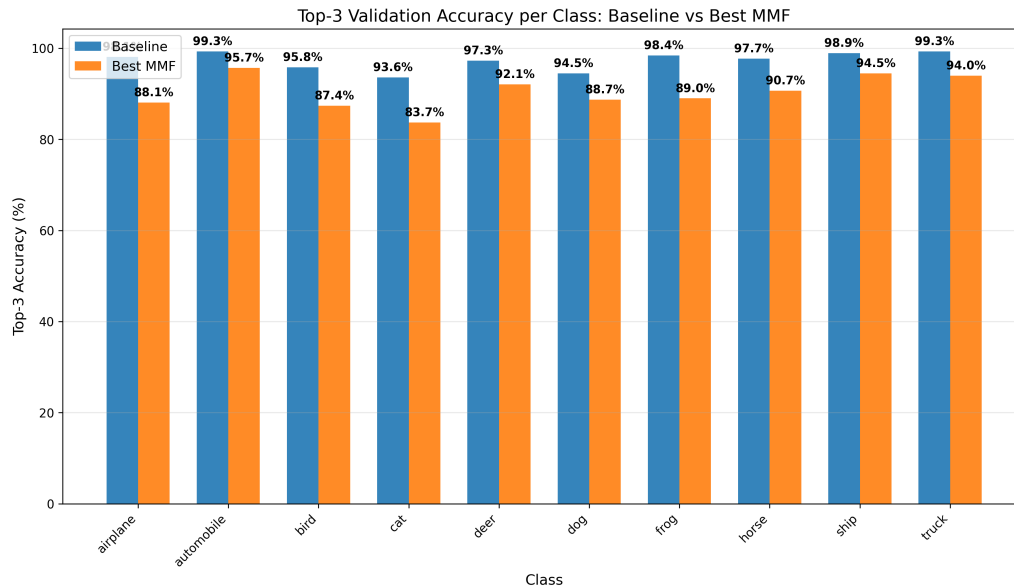


Figure 16: Baseline vs Best MMF Model Per Class Top-3 Accuracy

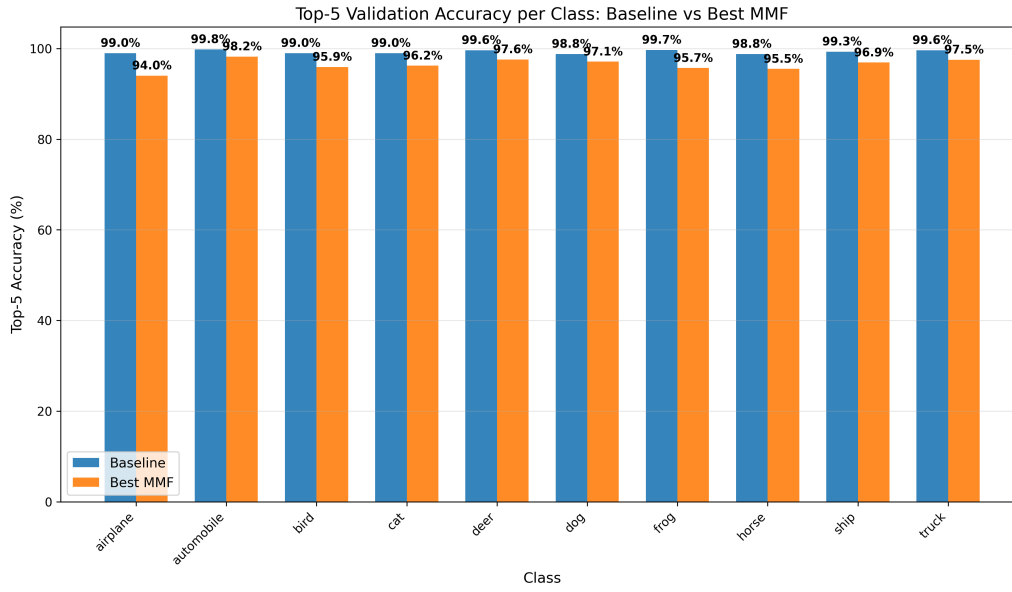


Figure 17: Baseline vs Best MMF Model Per Class Top-5 Accuracy

#### 4.4.5 Weight Distribution Comparison

Finally, the weight distribution of both models is plotted in Figure 18. The MMF Model is displayed by the raw and quantized weights, which represent respectively the weight values used during backpropagation at training, and the values used during the forward pass.

It can be observed that all distributions are symmetric. However, the distribution of the baseline model weights is closer to zero than the raw MMF Model, which is more evenly distributed between -0.5 and 0.5. Finally, when quantizing the weights of the MMF model, a lot of the raw weights fall to zero, surpassing the accumulated close to zero values of the baseline model.

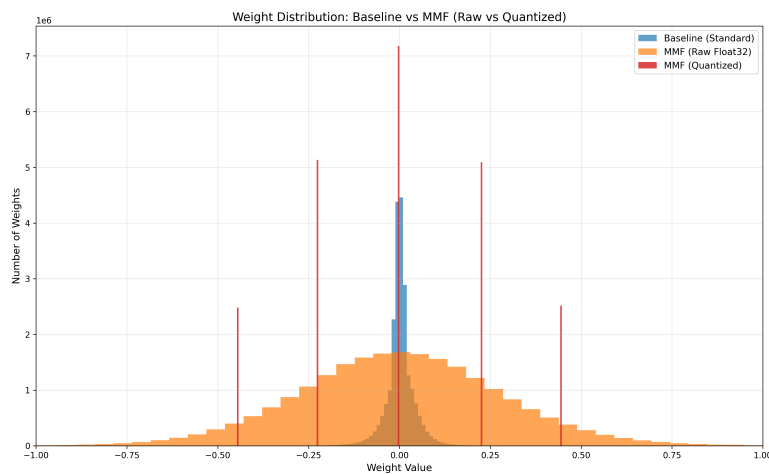


Figure 18: Weight Distribution for Baseline vs Best MMF Model

## 5 Conclusion

This project studied the feasibility of MatMul-Free (MMF) convolutional neural networks for image classification on CIFAR-10, comparing their performance against a full-precision baseline model under controlled experimental conditions.

First, the Baseline Model established a reference point for the CIFAR-10 dataset, achieving a best validation accuracy of 85.6%. The training process revealed extreme sensitivity to weight decay and learning rate. Data augmentation with normalization and the `ReduceLROnPlateau` scheduler proved essential for optimal performance.

Second, directly mapping the Baseline Model to MMF inference without retraining resulted in a complete collapse in performance, with validation accuracy dropping to 10.0% (random chance). This confirms that quantization-aware training is essential and that the learned representations of a full-precision model cannot be preserved under direct weight quantization.

Third, the MatMul-Free convolutional neural network has been shown to be trainable and capable of achieving non-trivial classification accuracy, which is a positive outcome of this project.

Forth, the gap between the MMF and the full-precision baseline reflects a fundamental representational bottleneck. The MMF Model trained with quantization-aware training reached a best validation accuracy of 67.1% under its optimal configuration (quinary weights with  $k = 2$ , weight initialization scale 0.2, learning rate  $3 \times 10^{-3}$ , and a cosine scheduler with the learning rate halved at  $T/2$ ), which represents an 18.5 percentage point gap relative to the Baseline Model.

Fifth, training experiments have shown that the MMF Model exhibited markedly different training dynamics, requiring a higher learning rate, zero weight decay, and a structured cosine annealing schedule rather than the adaptive plateau-based scheduler used for the Baseline.

Sixth, increasing the number of channels has demonstrated small validation accuracy improvements and higher computational costs, showing that the gap cannot be bridged by simply making the model larger.

Seventh, reducing the weight initialization scale improved validation accuracy considerably, making this modification the most effective intervention, but still leaving a substantial gap relative to the Baseline.

Eighth, increasing quantization levels from ternary ( $k = 1$ ) to quinary ( $k = 2$ ) weights provided marginal accuracy improvements, and further increases to septenary ( $k = 3$ ) and nonary ( $k = 4$ ) yielded no additional gains.

In conclusion, the 18.5 percentage point performance gap is primarily attributable to the fundamental representational constraint of quantized weights, and cannot be overcome by scaling model capacity, tuning hyperparameters, or incrementally increasing the number of discrete weight levels.

However, a clear baseline for MMF architectures is defined by this project, providing a rigorous experimental framework for future investigations.

Finally, closing the performance gap between full-precision and MatMul-free models remains an open challenge that will require further architectural innovation.

## 6 Future Work

This project establishes a foundation for MatMul-free convolutional models on image classification, and opens several directions for future investigation.

### Improving MMF classification accuracy

The most immediate next step is to continue closing the accuracy gap between the MMF and Baseline models. The experiments conducted in this project explore a subset of possible modifications, and several promising directions remain untested, including residual connections, alternative normalisation strategies, deeper architectures, and more sophisticated quantisation-aware training schedules. Reaching a classification accuracy competitive with the full-precision Baseline on CIFAR-10 would validate the MMF approach as a viable drop-in replacement for standard convolutional networks.

### Evaluation on dedicated MMF hardware

The computational advantages of MatMul-free inference (reduced memory bandwidth, elimination of floating-point multiplications, and ternary weight sparsity) are theoretical in the context of this project, as all experiments are conducted on standard GPU hardware that is not optimised for integer or ternary arithmetic. Once classification accuracy is sufficiently competitive, the MMF Model should be evaluated on dedicated hardware such as FPGAs or custom ASIC accelerators designed for low-bit inference. Key metrics to measure would include inference latency, energy consumption, and throughput, which are expected to be substantially lower than those of a full-precision model achieving the same accuracy.

### Extension to object detection

A natural extension of this work is to apply the MMF convolutional backbone to object detection. Following the approach of YOLOv1 [11], the classification backbone trained in this project could be used as a feature extractor, with a detection head attached to predict bounding boxes and class probabilities. This would allow a direct assessment of whether the accuracy trade-offs observed in classification transfer to the more challenging detection setting, and whether the efficiency gains of the MMF approach are preserved in a multi-task output regime.

### Adoption of state-of-the-art architectures

The MMF layers developed in this project are implemented within a relatively simple convolutional architecture. An important future direction is to integrate `MMFConv2d` and `MMFLinear` into state-of-the-art backbone architectures for both image classification and object detection, such as ResNets, EfficientNets, or Vision Transformers. This would provide a more rigorous benchmark of the MMF approach against modern full-precision models and allow the scalability of the method to be assessed on larger and more challenging datasets such as ImageNet or MS-COCO.

## Extension to other vision tasks

Beyond classification and detection, the MMF framework could be extended to more complex vision tasks such as semantic segmentation, instance segmentation, and pose estimation. These tasks require richer spatial representations and multi-scale feature extraction, making them a more demanding test of the expressiveness of ternary-weight networks. Success in these settings would broaden the applicability of the MMF approach and motivate its adoption in resource-constrained deployments such as edge devices and embedded systems.

## Hybrid Architectures

Finally, a promising direction for future research could be the exploration of hybrid architectures that selectively combine MMF layers with full-precision operations in critical parts of the network. Rather than applying ternary quantization uniformly across all layers, a hybrid approach would identify which layers benefit most from full-precision weights and retain floating-point arithmetic in those positions, while applying MMF quantization to the remaining intermediate layers. This strategy could preserve more representational capacity in the most sensitive parts of the network while still achieving substantial reductions in computational cost and memory bandwidth.

## References

- [1] Jared Kaplan et al. ‘Scaling Laws for Neural Language Models’. In: *arXiv preprint arXiv:2001.08361* (2020). URL: <https://arxiv.org/abs/2001.08361>.
- [2] Benoit Jacob et al. ‘Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference’. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018. URL: [https://openaccess.thecvf.com/content\\_cvpr\\_2018/html/Jacob\\_Quantization\\_and\\_Training\\_CVPR\\_2018\\_paper.html](https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html).
- [3] Matthieu Courbariaux, Yoshua Bengio and Jean-Pierre David. ‘BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations’. In: *Advances in Neural Information Processing Systems*. Vol. 28. 2015. URL: <https://proceedings.neurips.cc/paper/2015/hash/3e15cc11f979ed25912dff5b0669f2cd-Abstract.html>.
- [4] Yoshua Bengio, Nicholas Léonard and Aaron Courville. ‘Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation’. In: *arXiv preprint arXiv:1308.3432* (2013). URL: <https://arxiv.org/abs/1308.3432>.
- [5] Fengfu Li, Bo Zhang and Bin Liu. ‘Ternary Weight Networks’. In: *arXiv preprint arXiv:1605.04711* (2016). URL: <https://arxiv.org/abs/1605.04711>.
- [6] Hongyu Wang et al. ‘BitNet: Scaling 1-bit Transformers for Large Language Models’. In: *arXiv preprint arXiv:2310.11453* (2023). URL: <https://arxiv.org/abs/2310.11453>.
- [7] Shuming Ma et al. ‘The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits’. In: *arXiv preprint arXiv:2402.17764* (2024). URL: <https://arxiv.org/abs/2402.17764>.
- [8] Rui-Jie Zhu et al. *Scalable MatMul-free Language Modeling*. 2024. arXiv: 2406.02528 [cs.CL]. URL: <https://arxiv.org/abs/2406.02528>.
- [9] Alex Krizhevsky. ‘Learning Multiple Layers of Features from Tiny Images’. In: *Technical Report, University of Toronto*. 2009. URL: <https://www.cs.toronto.edu/~kriz/cifar.html>.
- [10] *What are convolutional neural networks?* 2025. URL: <https://www.ibm.com/think/topics/convolutional-neural-networks>.
- [11] Joseph Redmon et al. ‘You Only Look Once: Unified, Real-Time Object Detection’. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91. URL: <https://arxiv.org/abs/1506.02640>.
- [12] Mark Everingham et al. ‘The PASCAL Visual Object Classes Challenge: A Retrospective’. In: *International Journal of Computer Vision* 111.1 (2015), pp. 98–136. DOI: 10.1007/s11263-014-0733-5. URL: <https://link.springer.com/article/10.1007/s11263-014-0733-5>.
- [13] Jia Deng et al. ‘ImageNet: A Large-Scale Hierarchical Image Database’. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition* (2009), pp. 248–255. DOI: 10.1109/CVPR.2009.5206848.
- [14] Ekin D. Cubuk et al. ‘RandAugment: Practical Automated Data Augmentation with a Reduced Search Space’. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/d85b63ef0ccb114d0a3bb7b7d808028f-Abstract.html>.
- [15] Terrance DeVries and Graham W. Taylor. *Improved Regularization of Convolutional Neural Networks with Cutout*. 2017. URL: <https://arxiv.org/abs/1708.04552>.

- [16] Ilya Loshchilov and Frank Hutter. ‘Decoupled Weight Decay Regularization’. In: *International Conference on Learning Representations*. 2019. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [17] Ilya Loshchilov and Frank Hutter. ‘SGDR: Stochastic Gradient Descent with Warm Restarts’. In: *International Conference on Learning Representations*. 2017. URL: <https://openreview.net/forum?id=Hy7QRbcxl>.
- [18] Adam Paszke et al. ‘PyTorch: An Imperative Style, High-Performance Deep Learning Library’. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.



## A Code Repository Structure

The complete implementation of this project is available in a public GitHub repository<sup>3</sup>. The repository follows a modular structure that separates dataset preparation, model definitions, and training procedures, facilitating reproducibility and experimentation with different configurations. The code is organized as follows:

- **\_dataset/**: Contains the CIFAR-10 dataset files after extraction.
- **classification/**: Main project folder containing all source code and experimental runs.
  - **\_1\_dataset/**: Dataset preparation and management.
    - \* **samples/**: Visual samples from the dataset for verification.
    - \* **dataset.py**: Defines dataset splits (train/test) and data augmentation pipelines.
    - \* **test\_dataset.py**: Verification script that generates sample images to ensure correct dataset loading.
  - **\_2\_train/**: Training scripts, model definitions, and experimental results.
    - \* **runs/**: Stores all experimental runs with saved checkpoints, training logs, and performance metrics for different configurations tested throughout the project.
    - \* **layers\_mmf.py**: Implementation of custom MatMul-Free layers. Contains different layer variants explored during experimentation.
    - \* **model.py**: Defines both the Baseline Model and MMF Model architectures used in the comparative experiments.
    - \* **test.py**: Evaluation script for computing test set accuracy and generating performance metrics.
    - \* **train\_mmf.py**: Training pipeline for the MMF Model with quantization-aware training, learning rate scheduling, and checkpoint management.
    - \* **train.py**: Training pipeline for the Baseline Model with standard floating-point operations.
  - **\_3\_plot/**: Scripts used for plotting results displayed during this project report.
- **requirements.txt**: Lists all Python dependencies (PyTorch, NumPy, torchvision, etc.) required to reproduce the experiments.
- **README.md**: Project description for the github repository, including installation instructions and dataset setup.

---

<sup>3</sup>[https://github.com/jonurce/MatMulFree\\_Image\\_Classification\\_Model](https://github.com/jonurce/MatMulFree_Image_Classification_Model)

## B Complete Training Results

### B.1 Baseline Model: Hyperparameter Tuning

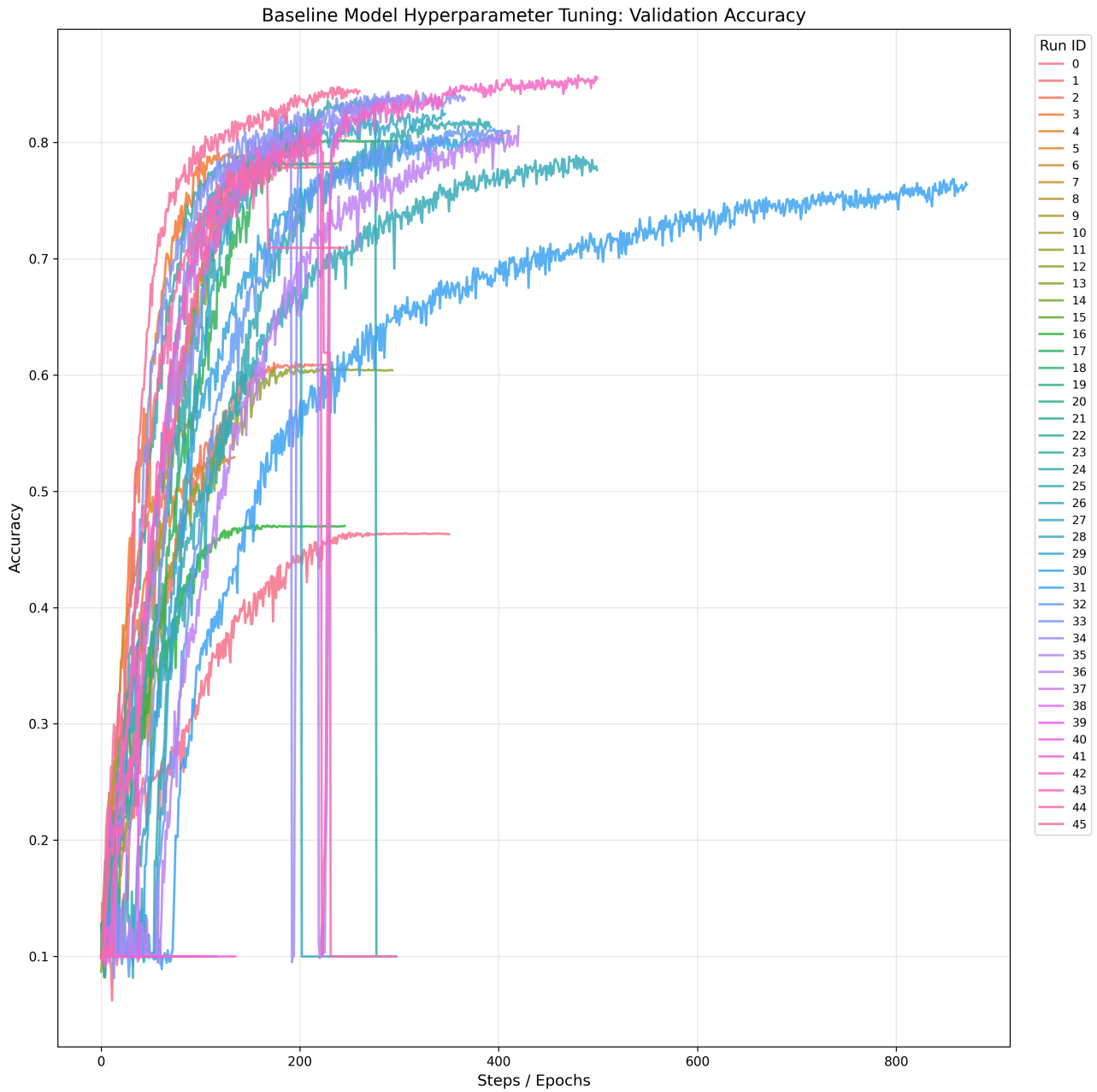


Figure 19: Validation Accuracy while Training for all Hyperparameter Tuning Experiments with Baseline Model

Table 14: Complete results from Baseline Model hyperparameter tuning. BS: batch size, LR: learning rate,  $\lambda$ : weight decay, Aug: augmentation level (W: weak, S: strong, S+N: strong with normalization), Sch: scheduler (ROP: ReduceLROnPlateau, OCL: OneCycleLR), Pat: patience.

| Run       | BS          | LR                                     | $\lambda$                              | Aug        | Pat       | Scheduler              | Train Acc    | Val Acc      |
|-----------|-------------|----------------------------------------|----------------------------------------|------------|-----------|------------------------|--------------|--------------|
| 0         | 2048        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 44.0%        | 46.3%        |
| 1         | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 87.4%        | 77.8%        |
| 2         | 1024        | $5.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 60.8%        | 60.8%        |
| 3         | 1024        | $2.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 88.9%        | 78.9%        |
| 4         | 512         | $1.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 51.0%        | 52.8%        |
| 5         | 512         | $1.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 55.0%        | 56.2%        |
| 6         | 512         | $9.9 \times 10^{-5}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 7         | 1024        | $3.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 8         | 1024        | $1.5 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 85.7%        | 78.2%        |
| 9         | 1024        | $1.5 \times 10^{-3}$                   | $1.0 \times 10^{-4}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 10        | 1024        | $2.0 \times 10^{-3}$                   | $1.0 \times 10^{-4}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 11        | 1024        | $2.0 \times 10^{-3}$                   | $1.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 12        | 1024        | $2.0 \times 10^{-3}$                   | $5.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 60.0%        | 60.4%        |
| 13        | 1024        | $2.0 \times 10^{-3}$                   | $7.0 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 14        | 1024        | $2.0 \times 10^{-3}$                   | $6.5 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 15        | 1024        | $2.0 \times 10^{-3}$                   | $6.1 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 16        | 1024        | $2.0 \times 10^{-3}$                   | $5.9 \times 10^{-5}$                   | W          | 15        | ROP: factor 0.5        | 46.3%        | 47.0%        |
| 17        | 1024        | $2.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 77.7%        | 80.1%        |
| 18        | 1024        | $2.0 \times 10^{-3}$                   | $6.1 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 19        | 1024        | $2.0 \times 10^{-3}$                   | $5.9 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 20        | 1024        | $2.0 \times 10^{-3}$                   | $5.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 21        | 1024        | $2.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 22        | 1024        | $2.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 23        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 73.6%        | 78.1%        |
| 24        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 84.0%        | 83.5%        |
| 25        | 1024        | $9.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 83.6%        | 81.3%        |
| 26        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | OCL                    | 76.7%        | 77.6%        |
| 27        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | OCL                    | 10.0%        | 10.0%        |
| 28        | 1024        | $6.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | OCL                    | 85.7%        | 82.4%        |
| 29        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | OCL                    | 10.0%        | 10.0%        |
| 30        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 83.1%        | 80.0%        |
| 31        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 15        | ROP: factor 0.5        | 74.4%        | 76.3%        |
| 32        | 1024        | $6.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 25        | ROP: factor 0.5        | 84.6%        | 80.9%        |
| 33        | 1024        | $8.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 25        | ROP: factor 0.5        | 87.3%        | 84.0%        |
| 34        | 1024        | $8.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 40        | ROP: factor 0.5        | 88.0%        | 83.7%        |
| 35        | 1024        | $9.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 25        | ROP: factor 0.5        | 86.1%        | 83.5%        |
| 36        | 1024        | $8.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 25        | ROP: factor 0.5        | 67.4%        | 81.4%        |
| 37        | 1024        | $6.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 25        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 38        | 1024        | $4.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S          | 25        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 39        | 1024        | $2.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S          | 25        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 40        | 1024        | $1.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S+N        | 25        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 41        | 1024        | $2.0 \times 10^{-3}$                   | $6.0 \times 10^{-5}$                   | S+N        | 25        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| <b>42</b> | <b>1024</b> | <b><math>8.0 \times 10^{-4}</math></b> | <b><math>6.0 \times 10^{-5}</math></b> | <b>S+N</b> | <b>25</b> | <b>ROP: factor 0.5</b> | <b>73.4%</b> | <b>85.6%</b> |
| 43        | 1024        | $8.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S+N        | 25        | ROP: factor 0.5        | 10.0%        | 10.0%        |
| 44        | 1024        | $8.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S+N        | 25        | ROP: factor 0.8        | 53.5%        | 70.9%        |
| 45        | 1024        | $8.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$                   | S+N        | 25        | ROP: factor 0.5        | 87.3%        | 84.0%        |

## B.2 MMF Model: Hyperparameter Tuning

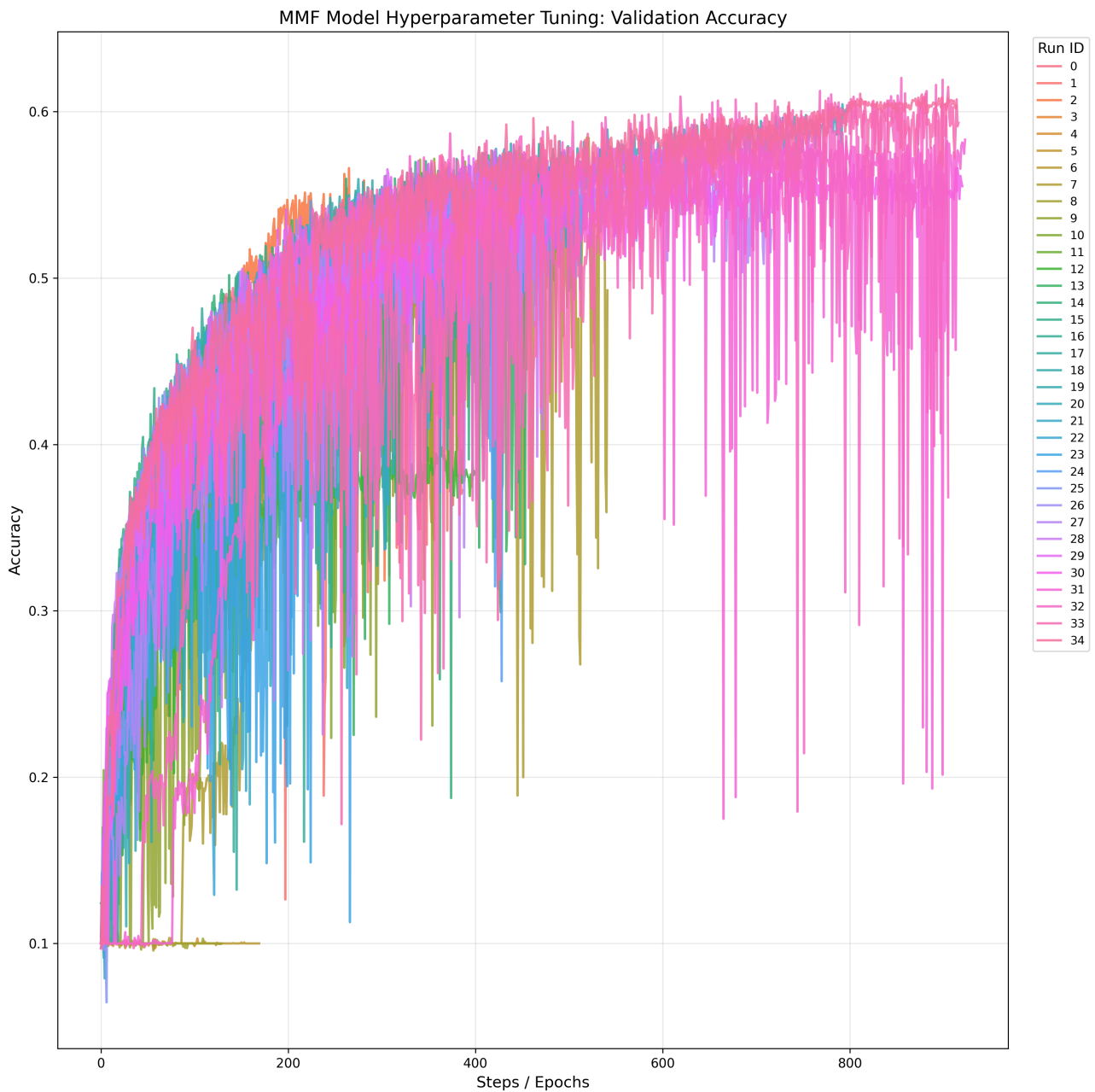


Figure 20: Validation Accuracy while Training for all Hyperparameter Tuning Experiments with MMF Model

Table 15: Complete results from MMF Model hyperparameter tuning (Runs 0–34). All runs use batch size 1024. LR: learning rate,  $\lambda$ : weight decay, Sch: scheduler (RLROP: ReduceLROnPlateau, Cos: CosineAnnealingLR).

| Run       | LR                                     | $\lambda$            | Max. Epochs | Scheduler                                    | Grad. Clip. | Train Acc    | Val Acc      |
|-----------|----------------------------------------|----------------------|-------------|----------------------------------------------|-------------|--------------|--------------|
| 0         | $2.0 \times 10^{-3}$                   | 0                    | 2000        | RLROP (fac. 0.5)                             | –           | 32.6%        | 45.9%        |
| 1         | $4.0 \times 10^{-3}$                   | 0                    | 2000        | RLROP (fac. 0.5)                             | –           | 36.7%        | 40.6%        |
| 2         | $6.0 \times 10^{-3}$                   | 0                    | 600         | Cos (min 0.1 $\eta$ )                        | –           | 32.6%        | 45.2%        |
| 3         | $8.0 \times 10^{-3}$                   | 0                    | 600         | Cos (min 0.1 $\eta$ )                        | –           | 10.0%        | 10.0%        |
| 4         | $5.0 \times 10^{-3}$                   | 0                    | 600         | Cos (min 0.1 $\eta$ )                        | –           | 10.0%        | 10.0%        |
| 5         | $8.0 \times 10^{-3}$                   | 0                    | 600         | Cos (min 0.1 $\eta$ )                        | –           | 10.0%        | 10.0%        |
| 6         | $6.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (min 0.1 $\eta$ )                        | –           | 10.0%        | 10.0%        |
| 7         | $6.0 \times 10^{-3}$                   | 0                    | 2000        | Cos (min 0.1 $\eta$ )                        | –           | 35.8%        | 45.8%        |
| 8         | $6.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (min 0.1 $\eta$ )                        | –           | 10.0%        | 10.0%        |
| 9         | $4.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (min 0.1 $\eta$ )                        | –           | 40.9%        | 54.8%        |
| 10        | $2.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (min 0.1 $\eta$ )                        | –           | 39.8%        | 51.8%        |
| 11        | $1.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (min 0.1 $\eta$ )                        | –           | 28.1%        | 39.6%        |
| 12        | $8.0 \times 10^{-4}$                   | $6.0 \times 10^{-5}$ | 2000        | RLROP (fac. 0.5)                             | –           | 27.7%        | 38.3%        |
| 13        | $4.0 \times 10^{-3}$                   |                      | 1000        | Cos (fac. 0.1)                               | 1.0         | 33.9%        | 47.2%        |
| 14        | $4.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (fac. 0.6)                               | 1.0         | 32.9%        | 46.9%        |
| 15        | $3.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (fac. 0.6)                               | 1.0         | 39.5%        | 53.2%        |
| 16        | $3.0 \times 10^{-3}$                   | 0                    | 1000        | Cos (min 0.1 $\eta$ )                        | 1.0         | 39.4%        | 51.8%        |
| 17        | $3.0 \times 10^{-3}$                   | 0                    | 500         | Cos (min 0.1 $\eta$ )                        | 1.0         | 42.9%        | 57.3%        |
| 18        | $3.0 \times 10^{-3}$                   | 0                    | 500         | Cos + half at $T/2$                          | 1.0         | 40.9%        | 54.5%        |
| 19        | $4.0 \times 10^{-3}$                   | 0                    | 500         | Cos + half at $T/2$                          | 1.0         | 43.5%        | 57.1%        |
| <b>20</b> | <b><math>3.0 \times 10^{-3}</math></b> | <b>0</b>             | <b>800</b>  | <b>Cos + half at <math>T/2</math></b>        | <b>1.0</b>  | <b>47.1%</b> | <b>60.2%</b> |
| 21        | $3.0 \times 10^{-3}$                   | 0                    | 1600        | Cos + half at $T/2$                          | 1.0         | 31.5%        | 46.3%        |
| 22        | $3.0 \times 10^{-3}$                   | 0                    | 1500        | Cos + half + Cos                             | 1.0         | 40.7%        | 55.4%        |
| 23        | $3.0 \times 10^{-3}$                   | 0                    | 1500        | Cos + Cos + Cos                              | 1.0         | 37.5%        | 51.8%        |
| 24        | $3.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + Cos + Cos                              | 1.0         | 39.6%        | 54.1%        |
| 25        | $3.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half + Cos                             | 1.0         | 40.6%        | 52.8%        |
| 26        | $3.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half at $T/3$ + flat                   | 1.0         | 39.1%        | 52.7%        |
| 27        | $3.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half at $T/3$ + flat                   | 1.0         | 41.4%        | 53.5%        |
| 28        | $3.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half at $T/3$ + flat                   | 1.0         | 38.4%        | 54.1%        |
| 29        | $3.0 \times 10^{-3}$                   | 0                    | 900         | Cos + half at $T/3$ + flat                   | 1.0         | 43.2%        | 57.3%        |
| 30        | $2.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half at $T/3$ + flat                   | 1.0         | 42.2%        | 55.6%        |
| 31        | $4.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half at $T/3$ + flat                   | 1.0         | 39.9%        | 57.9%        |
| 32        | $4.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half at $T/2$                          | 1.0         | 41.2%        | 47.9%        |
| 33        | $3.0 \times 10^{-3}$                   | 0                    | 1200        | Cos + half at $T/2$                          | 1.0         | 46.1%        | 59.3%        |
| <b>34</b> | <b><math>3.0 \times 10^{-3}</math></b> | <b>0</b>             | <b>1200</b> | <b>Cos + half at <math>T/3</math> + flat</b> | <b>1.0</b>  | <b>47.4%</b> | <b>60.3%</b> |

### B.3 MMF Model: Modifying Number of Channels

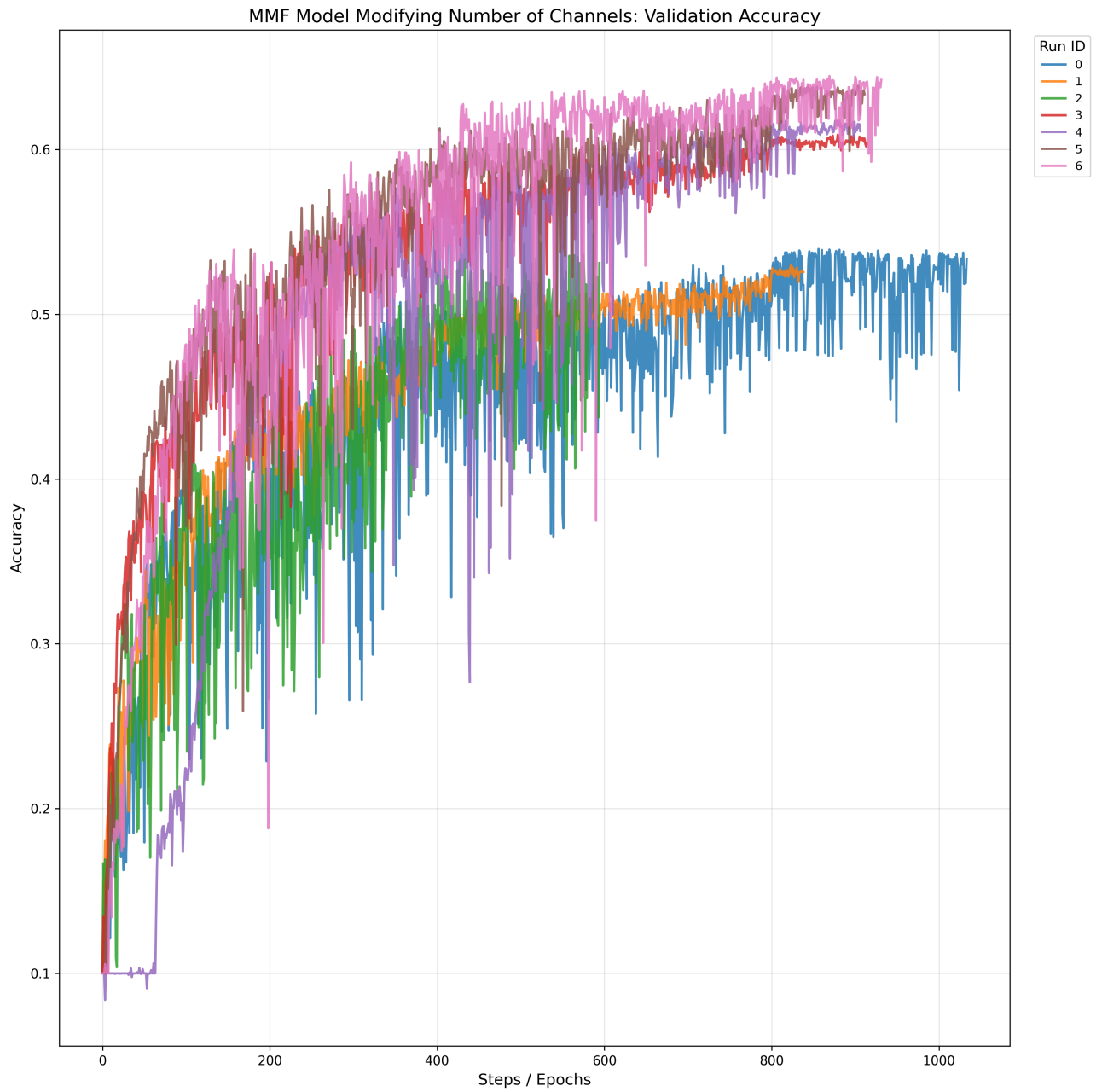


Figure 21: Validation Accuracy while Training for all Number of Channel Experiments with MMF Model

Table 16: Complete results for channel scaling experiments. The reference MMF Model ( $\alpha = 1.0$ , Run 20) is included for comparison.

| Run | $\alpha$ | LR                 | Params | GB/GPU | Time/epoch | Train Acc | Val Acc |
|-----|----------|--------------------|--------|--------|------------|-----------|---------|
| 0   | 0.5      | $3 \times 10^{-3}$ | 5.6M   | 3.7    | 7s         | 39.0%     | 52.7%   |
| 1   | 0.5      | $2 \times 10^{-3}$ | 5.6M   | 3.7    | 7s         | 40.0%     | 52.6%   |
| 2   | 0.75     | $3 \times 10^{-3}$ | 12.6M  | 5.3    | 12s        | 40.3%     | 53.1%   |
| 3   | 1.0      | $3 \times 10^{-3}$ | 20.0M  | 7.0    | 20s        | 47.1%     | 60.2%   |
| 4   | 1.5      | $4 \times 10^{-3}$ | 50.4M  | 10.5   | 36s        | 47.2%     | 61.3%   |
| 5   | 2.0      | $3 \times 10^{-3}$ | 92.0M  | 13.0   | 60s        | 50.5%     | 63.4%   |
| 6   | 3.0      | $3 \times 10^{-3}$ | 200.0M | 21.6   | 110s       | 49.7%     | 63.8%   |

## B.4 MMF Model: Modifying Weight Initialization Scale

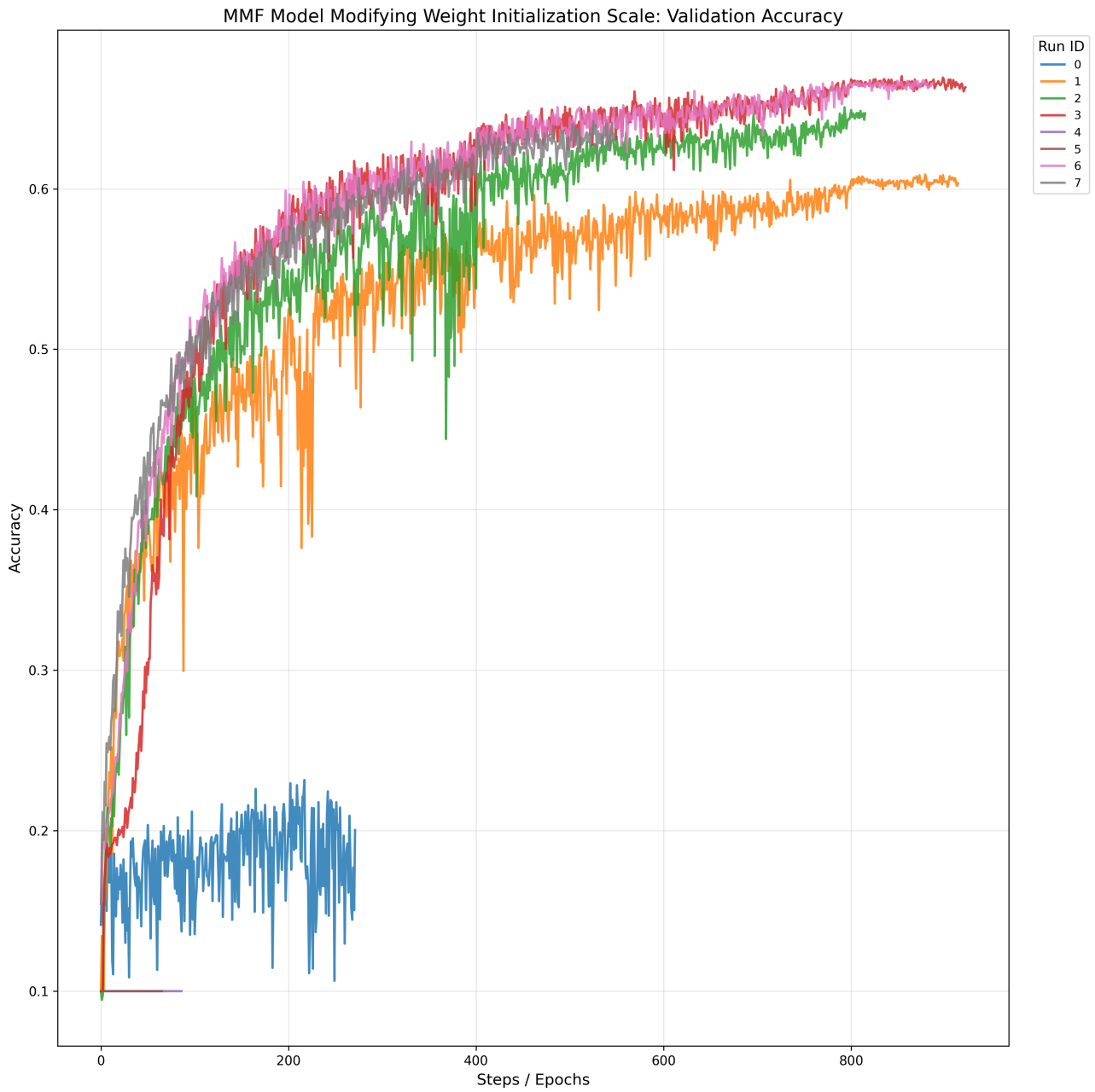


Figure 22: Validation Accuracy while Training for all Experiments Modifying Weight Initialization with MMF Model



Table 17: Complete results for weight initialization scale experiments. The reference MMF Model (scale = 1.0, Run 20) is included for comparison.  $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization.

| Run      | Scale      | LR                                   | mean( $ W $ ) | $W^-$        | $W^0$        | $W^+$       | Train Acc    | Val Acc      |
|----------|------------|--------------------------------------|---------------|--------------|--------------|-------------|--------------|--------------|
| 0        | 10.0       | $3 \times 10^{-3}$                   | 7.975         | 34.5%        | 31.0%        | 34.5%       | 14.7%        | 17.6%        |
| 1        | 1.0        | $3 \times 10^{-3}$                   | 0.839         | 34.4%        | 31.1%        | 34.5%       | 47.1%        | 60.2%        |
| 2        | 0.5        | $3 \times 10^{-3}$                   | 0.455         | 34.3%        | 31.2%        | 34.3%       | 51.3%        | 64.6%        |
| <b>3</b> | <b>0.2</b> | <b><math>3 \times 10^{-3}</math></b> | <b>0.247</b>  | <b>34.3%</b> | <b>31.3%</b> | <b>34.3</b> | <b>53.0%</b> | <b>66.3%</b> |
| 4        | 0.1        | $3 \times 10^{-3}$                   | 0.079         | 33.9%        | 31.2%        | 34.8%       | 10.0%        | 10.0%        |
| 5        | 0.1        | $2 \times 10^{-3}$                   | 0.079         | 34.1%        | 31.1%        | 34.7%       | 10.0%        | 10.0%        |
| 6        | 0.1        | $8 \times 10^{-4}$                   | 0.099         | 34.3%        | 31.2%        | 34.4%       | 52.6%        | 66.1%        |
| 7        | 0.1        | $6 \times 10^{-4}$                   | 0.092         | 34.3%        | 31.2%        | 34.4%       | 50.3%        | 63.2%        |

## B.5 MMF Model: Ternary, Quinary, Septenary, Nonary

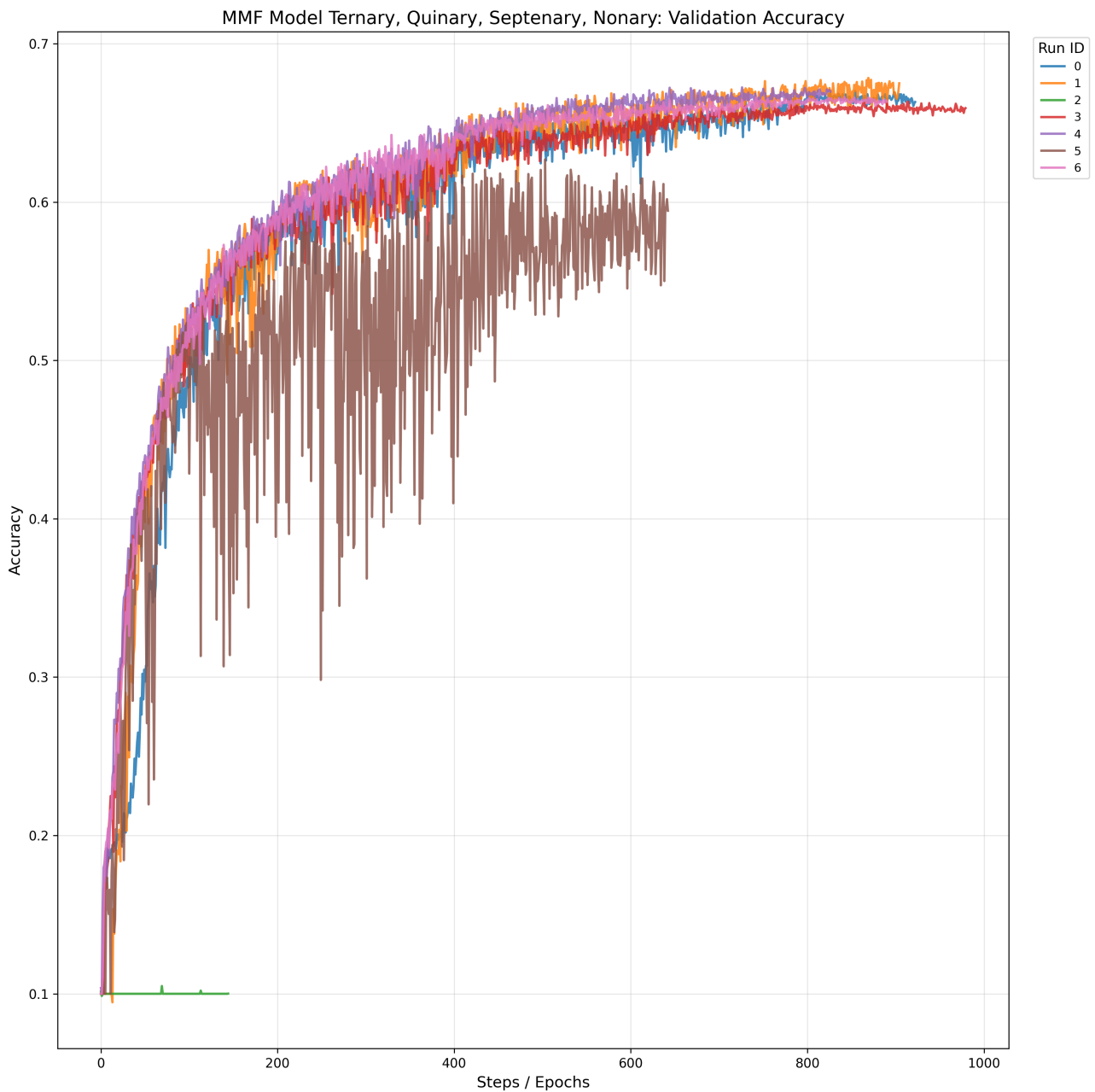


Figure 23: Validation Accuracy while Training for all Experiments for Ternary, Quinary, Septenary, Nonary MMF Models

Table 18: Complete results for quantization level experiments. The ternary reference ( $k = 1$ ) is included for comparison. W Init: weight initialization scale.  $W^-/W^0/W^+$ : percentage of negative, zero, and positive weights after quantization.

| Run      | k        | Levels   | $s_w$                                  | W Init     | mean( $ W $ ) | $W^-$        | $W^0$        | $W^+$        | Train Acc    | Val Acc      |
|----------|----------|----------|----------------------------------------|------------|---------------|--------------|--------------|--------------|--------------|--------------|
| 0        | 1        | 3        | $k/\text{mean}( W )$                   | 0.2        | 0.2475        | 34.3%        | 31.3%        | 34.3%        | 53.0%        | 66.3%        |
| <b>1</b> | <b>2</b> | <b>5</b> | <b><math>1/\text{mean}( W )</math></b> | <b>0.2</b> | <b>0.2624</b> | <b>34.2%</b> | <b>31.6%</b> | <b>34.2%</b> | <b>53.8%</b> | <b>67.1%</b> |
| 2        | 2        | 5        | $k/\text{mean}( W )$                   | 0.2        | 0.1630        | 34.0%        | 31.2%        | 34.8%        | 10.0%        | 10.0%        |
| 3        | 2        | 5        | $k/\text{mean}( W )$                   | 0.5        | 0.2624        | 34.2%        | 31.6%        | 34.2%        | 53.5%        | 65.9%        |
| 4        | 3        | 7        | $k/\text{mean}( W )$                   | 0.5        | 0.4326        | 34.3%        | 31.2%        | 34.5%        | 54.2%        | 67.0%        |
| 5        | 3        | 7        | $1/\text{mean}( W )$                   | 0.5        | 0.4644        | 34.3%        | 31.1%        | 34.6%        | 45.5%        | 59.4%        |
| 6        | 4        | 9        | $k/\text{mean}( W )$                   | 0.5        | 0.4298        | 34.3%        | 31.2%        | 34.5%        | 54.2%        | 66.4%        |